

Article

Drone Arc Routing Problems and Metaheuristic Solution Approach

Islam Altin * and Aydin Sipahioğlu

Department of Industrial Engineering, Eskisehir Osmangazi University, Eskisehir 26040, Turkey;
asipahi@ogu.edu.tr

* Correspondence: ialtin@ogu.edu.tr

Abstract: The drone arc routing problem (DARP) is one of the arc routing problems (ARPs) that has been studied by researchers recently. Unlike traditional ARPs, drones can travel directly between any two points in the graph. Due to the flexibility of drones, it is possible to use edges not defined in the graphs when deadheading the edges. This advantage of drones makes this problem more challenging than any other ARP. With this study, the energy capacities of drones are considered in a DARP. Thus, a novel DARP called the drone arc routing problem with deadheading demand (DARP-DD) is addressed in this study. Drone capacities are used both when servicing the edges and when deadheading the edges in the DARP-DD. A special case of the DARP-DD, called the multiple service drone arc routing problem with deadheading demand (MS-DARP-DD), is also discussed, where some critical required edges may need to be served more than once. To solve these challenging problems, a simulated annealing algorithm is used, and the components of the algorithm are designed. Additionally, novel neighbor search operators are developed in this study. The computational results show that the proposed algorithm and its components are effective and useful in solving the DARP-DD and MS-DARP-DD.

Keywords: capacitated arc routing problem; drone arc routing problem; deadheading demand; multiple service; simulated annealing algorithm



Citation: Altin, I.; Sipahioğlu, A. Drone Arc Routing Problems and Metaheuristic Solution Approach. *Drones* **2024**, *8*, 373. <https://doi.org/10.3390/drones8080373>

Academic Editors: Yu Wu and Liguu Sun

Received: 3 July 2024

Revised: 28 July 2024

Accepted: 29 July 2024

Published: 3 August 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Researchers have been interested in arc routing problems (ARPs) since the 18th century, and ARPs are employed in modern application areas. Drone-assisted operations are one of them [1,2]. With the advancement of drone technology, the application areas of drones have grown, and drones are now employed in a variety of areas [3]. The main application areas of drones are presented below [4]:

- Delivery of goods (products);
- Environmental mapping and monitoring;
- Traffic monitoring;
- Inspection of infrastructure, wind turbines, railway lines, energy transmission lines;
- Monitoring of forest areas and fire detection;
- Surveillance in search and rescue operations;
- Control and surveillance of regional borders in military operations.

As has been seen, there are difficult and dangerous tasks involved with the employment of drones. However, these tasks can be performed with drones quickly and effectively [5]. Drones offer superior surveillance, imaging, monitoring, and transmission capabilities due to the sensors, cameras, and communication systems that they have. For instance, forest areas can be monitored by drones, and the forest's condition can be evaluated [6]. Thus, precautions against the danger of a fire can be taken. In the case of a fire, drones may also be used to detect the fire quickly, and provide the opportunity to respond effectively. Moreover, drones contribute significantly to ensuring national security and conducting military operations [7], such that regional or national borders can be monitored and controlled by drones. So, drones are used to reveal potential dangers that may

pose a threat to the country's borders and to take the necessary precautions. In addition, reconnaissance can be made using drones to perform military intelligence operations [8].

Studies have been conducted to determine routes for drones in a significant and wide range of application areas. However, the drone routing problem is generally considered a node routing problem in the literature. For example, in a recent study conducted by Lu et al. [9], optimizing takeout delivery using drones and riders jointly was addressed. A two-stage heuristic algorithm was proposed to optimize order allocation and routing. Eskandaripour et al. [10] and Coutinho et al. [11] presented a literature review about drone routing studies and provided detailed information about them. In the node routing problem, the points (customers) that need to be serviced are located at the nodes [12]. On the other hand, in the arc routing problem, the points (customers) that need to be serviced are located on the arcs (edges) [13].

The tasks that drones are required to perform were defined on the edges by Campbell et al. [14], and the drone arc routing problem (DARP) was introduced to the literature [1,15]. The main difference between the DARP and the known ARPs is the ability of the drones to travel directly between two points (nodes) without following the edges in the graph. Due to this ability of drones, the chance to traverse edges that are not defined in the graph occurs while deadheading the edges with drones. If a vehicle (drone) travels through an edge without performing a task, that edge is referred to as a deadheading edge [16]. Therefore, the distance between the deadheading edges is the Euclidean distance between the two points. Considering all of these, the DARP is described as obtaining the minimum cost tour to service all the required edges with a single drone. In other words, the stated DARP is the rural postman problem (RPP) with some assumptions [17].

Campbell et al. [18] focused on a DARP with a length constraint for drones, and the length-constrained K-drones rural postman problem (LC K-DRPP) was introduced to the literature. In this problem, the drone route is limited to a maximum length, necessitating the use of multiple drones to service the required edges. Similar to the DARP, after finishing the service of a required edge, the drone can travel directly to any vertex of another required edge without following the edges of the graph. Thus, this feature provides flexibility and efficiency in route planning. Moreover, the LC K-DRPP has the main features of the DARP suggested by Campbell et al. [14].

In this study, surveillance drones used in search and rescue operations, control of regional borders, inspection of infrastructure, wind turbines, etc., are discussed. Surveillance drones travel from the depot (central point) to the areas that need to be serviced and the borders in the relevant areas are monitored or inspected [19]. Drones use sensors and cameras to perform monitoring and inspection tasks. With their limited energy capacity, the drones should go to the areas that need to be served, perform monitoring tasks in those regions, and then return to the depot. That is to say, drones must perform their tasks and complete their routes without exceeding their energy capacity [4]. There is an important issue to be emphasized about the energy consumption of drones. While surveillance drones do not serve the edges, they only consume as much energy as the traveled distance; when serving the edges, they consume as much energy as demanded by both the traveled distance and the sensors, imaging, and communication devices (avionics) activated during surveillance [20–22]. Namely, drone capacities are used both when serving the edges and deadheading the edges. The related problem is called the drone arc routing problem with deadheading demand (DARP-DD). With this study, a DARP-DD considering the energy capacity of drones is addressed, and all required edges are served by drones only once in DARP-DD. Moreover, a special case may be encountered in the DARP-DD investigated. In some special cases, it may be necessary to provide multiple services to some required edges. This need can arise due to several reasons, such as ensuring more precise imaging of the relevant edge due to its importance, maintaining security for strategic or critical areas, improving data accuracy and reliability by collecting data from the same area multiple times, and taking preventive measures by frequently monitoring certain areas. The prob-

lem encountered in this case is called the multiple service drone arc routing problem with deadheading demand (MS-DARP-DD).

The DARP-DD and MS-DARP-DD have real-life applications. For example, the parts that need to be monitored in forest areas can be represented by edges. In this case, the monitoring operations of the edges described in the forest area by energy-capacitated surveillance drones are an application area of the DARP-DD. On the other hand, in cases where some strategic and critical edges need to be monitored more than once to ensure the security of the forest area, drones are expected to respond to this demand. In this case, the application area of the MS-DARP-DD is encountered. The MS-DARP-DD enhances security measures in strategic areas and facilitates proactive monitoring of sensitive locations. The mentioned application area can be expanded as traffic monitoring by representing traffic roads in a city with edges, inspection of railway lines by representing rails with edges, and surveillance of regional borders by representing country borders with edges.

The simulated annealing algorithm [23,24] is proposed to solve these two novel problems in this study. This algorithm has many components, which include solution representation, neighbor search operators, constraint handling strategy, and parameter values such as initial temperature value, final temperature value, cooling rate, and the number of iterations. So, the components of the algorithm are designed in accordance with these problems. The proposed algorithm is initially applied to solve the capacitated arc routing problem (CARP), which is known as one of the most challenging ARPs belonging to the NP-hard class. Thus, the success of the proposed algorithm in solving ARPs has been proven by using the most challenging problems in the literature. In the continuation of this study, the proposed algorithm is then utilized to solve the test problems generated for the DARP-DD and MS-DARP-DD. The performance of the algorithm is analyzed in detail. As previously stated, the problems addressed in this study have many real-life counterparts in applications. As a result, the solution approach developed in this study will benefit these areas. It has significant implications for practical applications. For instance, the routes obtained for the drones provide efficient resource utilization and increase operational efficiency in various sectors that rely on drone-based surveillance and inspection missions.

The remainder of this paper is organized as follows: In Section 2, the literature related to the DARP is reviewed. In Section 3, we give descriptions of the DARP-DD and MS-DARP-DD. Section 4 gives detailed information about the proposed approach. Section 5 presents the computational results. In Section 6, we conclude our study and make suggestions for future studies.

2. Related Work

The DARP was first described by Campbell et al. [14] in the literature. But in fact, there are arc routing problems in the literature that are suitable for the characteristics of a DARP. Therefore, information about these problems is presented.

Garfinkel and Webb [25] defined the crossing postman problem (XPP) as a general form and application of the undirected rural postman problem (RPP). In the XPP, the postman is allowed to enter or exit from any point of an edge in the undirected and disconnected graph. As a potential application of the XPP, Garfinkel and Webb [25] gave an example of obtaining the minimum distance tour for a machine that cuts polygonal shapes from a material such as fabric or metal. In the following years, cutting path problems (CPPs) were studied as an application of the XPP in the literature.

Dewil et al. [26] conducted a literature review about cutting path problems and this problem was classified. The endpoint cutting problem (ECP), continuous cutting problem (CCP), and intermittent cutting problem (ICP) are among these problems, sharing common characteristics with the DARP. The ECP is the problem of cutting shapes from a material so as to minimize the total cost (distance) by using predetermined entry and exit points on the shapes. In the CCP, each shape must be cut at once without dividing it into edges. Additionally, it is not necessary to use the predetermined points on the shapes; cutting can

be started from any point of the shape. In the ICP, shapes can be cut into pieces and there is no restriction on the entry or exit points to be used.

Campbell et al. [14] defined the DARP in 2018, but the energy capacities of drones are not taken into account in the study. Therefore, the demands of the required edges on the graph can be met by using a single drone. That is why the DARP addressed in the study is actually a special case of the rural postman problem (RPP). They proposed an algorithm to solve the DARP. The proposed algorithm iteratively solves RPP instances with an increasing number of segments. For the solution of the RPP in each step, the branch and cut algorithm proposed by Corberán et al. [27] for the windy general routing problem was applied.

Campbell et al. [18] and Campbell et al. [28] focused on a DARP with a length constraint for drones in 2021 and 2022. That is why this problem is called the length-constrained K-drones rural postman problem (LC K-DRPP). The LC K-DRPP is more challenging than the DARP since it is necessary to service the required edges with multiple drones. However, the energy capacity of drones was not taken into account in the study. Campbell et al. [18] a proposed branch-and-cut algorithm for the length-constrained K-vehicles rural postman problem (LC K-RPP) and matheuristic algorithms to solve the LC K-DRPP. In 2022, Campbell et al. [28] carried out a polyhedral study and presented a new sophisticated branch-and-cut algorithm for the LC K-RPP. They also developed a global algorithm for the LC K-DRPP that iteratively solves instances of the LC K-RPP using the proposed branch-and-cut algorithm.

Xie et al. [17] addressed the drone rural postman problem, where there is a single drone without an energy capacity constraint. They proposed a two-stage heuristic algorithm to solve the problem. This algorithm firstly reduces the graph and then solves the RPP on the reduced graph. The proposed two-phase heuristic was compared with the solution approach suggested by Campbell et al. [14], and it was shown that the two-phase heuristic performed better.

Many researchers have highlighted the importance of drones' energy consumption features. Abubakar et al. [29] presented a detailed survey on energy optimization techniques in unmanned aerial vehicle (UAV)-based cellular networks. They discussed four different types of energy optimization in UAV-based cellular networks: optimization of the propulsion energy, optimization of the communication energy, joint optimization, and optimization of the energy consumption in UAV-assisted cellular networks. Then, they reviewed conventional (exact, heuristic, and metaheuristic methods) and machine learning algorithms to solve these energy optimization problems. Cabuk et al. [20] focused on the energy consumption patterns of small drones in their study. They conducted experimental study using a small drone to collect its energy consumption data. They developed four different models for predicting the drone's energy consumption patterns, which include a theoretical model, a simple linear regression model, a cubic polynomial regression model, and a machine learning-based model. It is noted that the drone could consume more energy due to its avionics, but this issue was not taken into account in the study. In addition, it was stated in the studies by Luo and Chen [21] and Zhang et al. [22] that drones consume more energy while servicing the edges. Campbell et al. [14] stated that battery consumption may be proportional to the activities performed by the drone. Additionally, they reported that the drones' battery consumption can increase when servicing the edges.

A structure similar to the issue of energy consumption encountered in surveillance drones was described in the literature. Kirlik and Sipahioglu [16] introduced the capacitated arc routing problem with deadheading demand to the literature. In this study, they addressed the multi-robot coverage problem. The problem indicated that even if the robot passes through a path without performing a task (deadheading), it consumes its remaining energy and when it performs a task, it consumes more energy due to the activation of the cameras or sensors on it. They proposed a mathematical model as an exact solution method to solve the small-sized instances. Moreover, they developed a constructive heuristic method to solve the medium- and large-sized instances.

Examining the studies in the literature reveals that the DARP is considered a special version of the RPP. However, the problems addressed in this study is a special case of the CARP. Thus, in this study, drone routes were obtained to perform edge monitoring and inspection tasks with drones while taking into account their energy capacity, which is highlighted in the literature as a significant issue. Therefore, it is clear that considering the energy capacities of the drones makes the problem more realistic and applicable to real-world scenarios. This study also discusses a novel case of the DARP-DD, where some critical areas or borders need to be monitored more than once with surveillance drones. This special case (MS-DARP-DD) is encountered in real-life applications, especially when strategic or critical areas need to be monitored repeatedly in order to ensure the security of these areas. This work also presents novel neighbor search operators, which have been demonstrated to be useful in ARPs and are easily applicable to the structure of metaheuristic algorithms.

3. Problem Definition

There are various types of drones based on their usage areas. This study focuses on surveillance drones. Surveillance drones travel from a central point (depot) to the areas and the borders in these areas are monitored. Monitoring processes are performed by the sensors or monitoring devices on drones. With their limited energy capacity, drones should be able to travel to the areas that need to be serviced, capture images of these service areas (areas to be monitored) using sensors or monitoring devices, transfer the images simultaneously with a communication mechanism if necessary, and then return to the central point. The characteristics of the DARP-DD investigated due to the unique structure observed in surveillance drones is presented below:

- Drones have limited energy capacity.
- Drones consume energy both when servicing the edges and when deadheading the edges (passing without servicing).
- Drones consume more energy while servicing.
- The demand value of the deadheading edges is equal to the amount of energy consumed by the drone during deadheading.
- The demand value of the (required) edges that need to be serviced on the graph is equal to the amount of energy consumed by the drone while servicing.
- After the drone has completed servicing a required edge, it can directly travel to the node of any other required edge on the graph [14]. This distance value is equal to the Euclidean distance between the two nodes.
- The required edges on the graph can be linear or curved.

Considering all these features, the DARP-DD is defined as an arc routing problem in which k drones start and finish their tours at the depot node to minimize the total tour distance (cost) and ensure that the energy capacity of the drones is not exceeded. Therefore, the objective function of the DARP-DD is to minimize the total distance traveled by the drones. Figure 1 shows the graph of an example generated for the DARP-DD. Through this graph, how the problem is addressed is explained.

In the graph shown in Figure 1, there are 5 components, a depot where drones are available to service the required edges, 17 required edges that need to be serviced, and 19 nodes including the depot node. $E_R = \{(1-2), (1-3), (3-2), (11-12), (6-4), (6-5), (5-4), (7-8), (8-10), (10-9), (9-7), (13-14), (14-15), (15-16), (16-17), (17-18), (18-13)\}$ is the set of the required edges on the graph. The coordinates of the nodes on the graph are presented in Table 1.

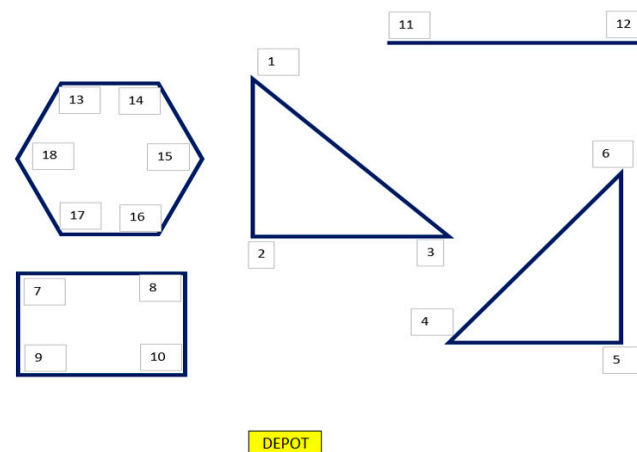


Figure 1. DARP-DD example 1.

Table 1. Coordinate values of DARP-1 nodes.

Nodes	Coordinate Values (x, y)
Depot	(0, −110)
1	(0, 0)
2	(0, −40)
3	(30, −40)
4	(30, −80)
5	(60, −80)
6	(60, −20)
7	(−60, −50)
8	(−20, −50)
9	(−60, −90)
10	(−20, −90)
11	(20, 10)
12	(70, 10)
13	(−50, 0)
14	(−20, 0)
15	(−10, −20)
16	(−20, −40)
17	(−50, −40)
18	(−60, −20)

On the graph presented in Figure 1, the distance values of the required edges can be calculated using the Euclidean distances of the nodes belonging to the edges. For this reason, the distance values of the edges are calculated using the coordinate values of the nodes presented in Table 1. For example, the distance value of the (1 – 2) edge is equal to 40 units, which is the Euclidean distance between the points (0, 0) and (0, −40). This unit provides a more flexible framework for distance measurement, and it can be interpreted in meters, kilometers, or miles. To determine the energy consumption of drones, the energy per meter (epm) metric proposed by Zhang et al. [22] for drones was considered. The epm was defined as a value that measures energy consumption per unit distance and used as energy per unit distance in this study. This way, the metric is made applicable to various scenarios. It is assumed that drones consume 1 Joule per unit distance (1 Joule/distance) during deadheading. On the other hand, drones consume 2 Joules per unit distance (2 Joules/distance) during servicing. This can also work with different values and the energy per unit distance can be adjusted based on drone type, usage purpose, or environmental conditions. On the other hand, the amount of energy consumed by drones when servicing the edges is not necessarily proportional to the distance traveled. Some edges need more detailed monitoring, and more energy is consumed, while others need less detailed monitoring, and less energy is consumed when servicing them. In this problem,

the drone consumes 40 Joules when deadheading through an edge (1 – 2) and 80 Joules when servicing the same edge. Thus, the deadheading demand value for an edge (1 – 2) is 40 Joules, while the servicing demand value is 80 Joules. Please note that the non-required edges only have a deadheading demand value, because drones do not serve these edges.

The mobility of the drone on the graph is determined as follows: The drone can directly traverse from the depot node to all nodes on the graph and from all nodes to the depot node. The edges where the drone can traverse from the depot to other nodes are shown with red dashed lines in Figure 2.

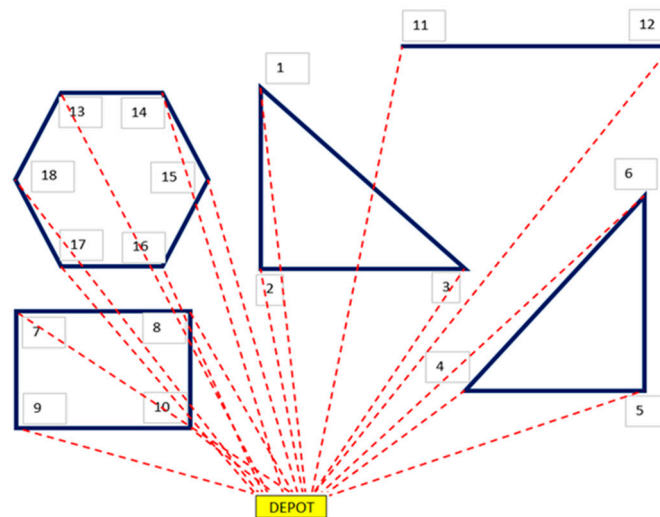


Figure 2. Traversing from the depot node to the other nodes in the DARP-DD.

As presented in Figure 2, the drone can directly traverse any node on the graph from the depot node by deadheading. On the other hand, the drone can directly traverse between all predefined nodes on the graph. In other words, after the drone has completed servicing the required edge, it can directly traverse to the node of any other required edge on the graph. The mobility of the drone between nodes is presented in Figure 3. Since the drone can perform a large number of deadheading between nodes, only a subset of them are presented in Figure 3.

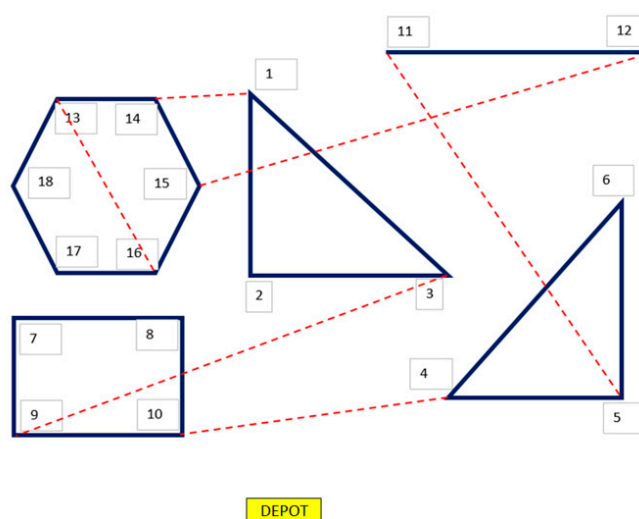


Figure 3. Traversing between nodes in the DARP-DD.

In Figure 3, the deadheading edges that drones can traverse between nodes are shown by the red dashed lines. For example, the drone can carry out deadheading between different components with edges such as (15 – 12), (9 – 3), and (14 – 1) as well as between

nodes of the same component such as the edge (13 – 16). Since the drone is capable of traversing between all nodes, the example consists of 19 nodes in total including the depot node and 17 required edges. The number of non-required edges in the graph is calculated by the number of nodes. The disconnected graph is converted to complete graph since the drone can directly traverse between all nodes on the graph. So, this graph will have $(n \times (n - 1)) / 2$ non-required edges with n nodes. As a result, this example has 171 non-required edges.

In the relevant problem, some critical areas or borders may need to be monitored multiple times by drones during surveillance. In this case, servicing the required edges that correspond to the relevant boundaries (areas) only once is not sufficient.

So, we can generate an instance for the MS-DARP-DD using Figure 1. For example, on the graph presented in Figure 1, due to the special case of the boundaries represented by the edges (2 – 3) and (9 – 10), it may be necessary to monitor the required edge (2 – 3) three times and the required edge (9 – 10) two times. The related multiple services to the required edges can be performed by the same drone or different drones.

4. Proposed Solution Approach

The shortest distances and the shortest paths between all pairs of nodes on the graph are first calculated using the Floyd–Warshall algorithm. After this phase, the simulated annealing algorithm is used to solve the problems. The simulated annealing algorithm was independently developed and introduced to the literature by Kirkpatrick et al. [23] and Černý [24]. The SA is a stochastic neighborhood search method that imitates the physical annealing process where a solid is slowly cooled until the minimum energy state is achieved. Therefore, not only improving solutions are always accepted, but also non-improving (inferior) solutions can be accepted by probability acceptance of the Metropolis criterion. This probability acceptance function is the main mechanism of the SA algorithm, and this function prevents getting stuck into local optima. Because of its simplicity and efficiency, the SA algorithm has been widely applied to solve most combinatorial optimization problems.

In this study, the proposed solution approach and its components are designed to be highly flexible, making them easily applicable to solving different ARPs. The solution representation was constructed in a dynamic manner, allowing for its size to be longer and shorter during the search process. This enables the effective use of various neighbor search operators. Furthermore, the algorithm uses 8 different neighbor search operators. Among these, the k -route shorten, smart edge direction change, and smart reverse operators are novel. Furthermore, the edge postpone operator is used in combination with the merge and split procedures, which is another aspect of the innovation of the proposed algorithm.

Under this heading, detailed explanations of the algorithm's components, including the solution representation, neighbor search operators and constraint handling strategy used in the structure of the algorithm are explained. Additionally, the working mechanism of the algorithm and its designed components are presented.

4.1. Solution Representation and Initial Solution

The solution representation is constructed as a two-dimensional list based on the required edges in the graph. However, when using the neighbor search operators, the deadheading edges is also included in the solution representation. Therefore, the solution representation to be used in the structure of the algorithm is dynamic. The proposed solution representation is explained using the graph presented in Figure 1. $E_R = \{(1 - 2), (1 - 3), (3 - 2), (11 - 12), (6 - 4), (6 - 5), (5 - 4), (7 - 8), (8 - 10), (10 - 9), (9 - 7), (13 - 14), (14 - 15), (15 - 16), (16 - 17), (17 - 18), (18 - 13)\}$ is the set of the required edges to be visited on the graph. Assume that the 4 drones' routes serving the required edges are as shown in Figure 4.

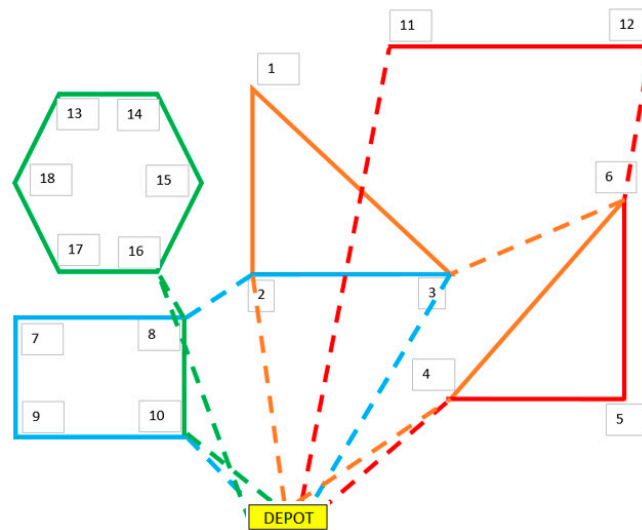


Figure 4. Sample drone routes.

In Figure 4, each drone route is represented by a different color. The dashed lines indicate the deadheading edges. Therefore, the resulting drone routes can be expressed as follows (d indicates the deadheading on the relevant edge):

First drone route (blue-colored route): $d(\text{depot} - 10) - (10 - 9) - (9 - 7) - (7 - 8) - d(8 - 2) - (2 - 3) - d(3 - \text{depo})$.

Second drone route (red-colored route): $d(\text{depot} - 4) - (4 - 5) - (5 - 6) - d(6 - 12) - (12 - 11) - d(11 - \text{depot})$.

Third drone route (green-colored route): $d(\text{depot} - 10) - (10 - 8) - d(8 - 16) - (16 - 15) - (15 - 14) - (14 - 13) - (13 - 18) - (18 - 17) - (17 - 16) - d(16 - \text{depot})$.

Forth drone route (orange-colored route): $d(\text{depot} - 2) - (2 - 1) - (1 - 3) - d(3 - 6) - (6 - 4) - d(4 - \text{depot})$.

It is seen that all 17 required edges are serviced. The shortest paths on the route are expressed as d, which stands for deadheading. Since the drone routes are included as a two-dimensional list based on the required edges in the solution representation, the solution representation corresponding to the obtained drone routes is as follows:

$\{((10 - 9) - (9 - 7) - (7 - 8) - (2 - 3)), ((4 - 5) - (5 - 6) - (12 - 11)), ((10 - 8) - (16 - 15) - (15 - 14) - (14 - 13) - (13 - 18) - (18 - 17) - (17 - 16)), ((2 - 1) - (1 - 3) - (6 - 4))\}$

We can define the edges in the solution representation as $Route_{ij}$: the j th order edge of the i th drone route. For example, edge (12 - 11) is the third-order edge of drone route 2 and can be described as $Route_{23}$. On the other hand, the solution representation does not have to contain only the required edges. This is because the information about the non-required edges in the route is also used in the generating neighbor solution phase. In this case, it is also necessary to include the deadheading edges in the solution representation. The length of the solution representation could increase or decrease. Thus, the length of the solution representation is dynamic.

The initial solution was randomly generated with the proposed solution representation. No problem-specific heuristic algorithm in order to construct initial solution was implemented.

4.2. Neighbor Search Operators

In the development of the neighbor search operators, we took advantage of the merge and split operator, which is one of the local search operators suggested by Tang et al. [30] for the CARP, and the shorten procedure, which is one of the post-optimization procedures for the rural postman problem suggested by Hertz et al. [31]. They proposed the shorten

procedure to improve the obtained routes for the RPP. A shorten procedure consists of the reverse and postpone mechanisms.

Eight different neighbor search operators are described to apply in the structure of the algorithm. In each iteration of the algorithm, one of these operators will be selected randomly. Each operator has an equal chance of selection.

K-Reverse Operator: A randomly selected k number of drone routes are merged. The value of k is 2 or 3. The merging process is performed according to the ascending route index and the routes are merged to obtain a single route. Two position values are randomly generated and the set of edges between these two positions is reversed in the single route. The obtained route is divided into k routes based on the drone capacities.

N-Reverse Operator: All drone routes are merged based on the ascending route index to obtain a giant route. Two position values are randomly generated and the set of edges between these two positions is reversed in the giant route. The obtained route is divided into n routes based on the drone capacities.

Intra-Route Reverse Operator: A route is randomly selected to change the order and direction of the edges on the route. Two position values are generated and the set of edges between these two positions is reversed to generate the neighbor solution.

Smart Reverse Operator: Two randomly selected routes are merged based on their index values in ascending order to obtain a single route. One of the repeating nodes in the merged route is randomly selected. The edges between the selected repeating nodes are reversed according to the defined priority rules. Four priority rules have been described. Whichever rule the repeating node first complies with, the related reverse process is performed in accordance with the rule. Assume that the repeating node is node i .

- Rule 1: If the route includes $(i - j) \dots (k - i)$, reverse it to $(i - k) \dots (j - i)$.
- Rule 2: If there are $(i - j) \dots (i - k)$ on the route, first change the direction of the edge $(i - k)$ to $(k - i)$. Then reverse it to $(i - k) \dots (j - i)$.
- Rule 3: If there are $(j - i) \dots (k - i)$ on the route, first change the direction of the edge $(j - i)$ to $(i - j)$. Then reverse it to $(i - k) \dots (j - i)$.
- Rule 4: If there is $(j - i) \dots (i - k)$ on the route, change the direction of both edges. Then reverse it to $(i - k) \dots (j - i)$.

The obtained route after the reverse process with one of the presented rules is divided into two based on the drone capacities and thus the neighbor solution is generated.

K-Route Shorten Operator: A randomly selected k number of drone routes are merged. The value of k is determined randomly between 2 and 5. In addition, k cannot have a value greater than the number of drones. The edges in the k number of selected routes are added to an edge pool and the nearest neighbor algorithm is used to generate a tour from the edges. The obtained tour is divided based on drone capacities to obtain k number of drone routes. The steps of the k -route shorten algorithm are presented below.

- Step 1: Add all the edges in the selected routes to an edge pool and create an empty route.
- Step 2: Obtain a route using the nearest neighbor algorithm with the edges in the edge pool.
 - a. Randomly select one of the edges nearest to the depot node from the edge pool, remove it from the edge pool, and add it to the route as the first edge to be visited in the route.
 - b. Randomly select one of the edges nearest to the last edge in the route from the edge pool and add it to the route by removing it from the edge pool. Repeat this step until there are no more edges in the edge pool.
- Step 3: Divide the obtained route into k number of routes based on the drone capacities.

Edge Postpone Operator: A randomly selected k number of drone routes are merged based on their index values in ascending order to obtain a single route. The value of k is determined randomly between 2 and 5. In addition, k cannot have a value greater than the number of drones. Then, the deadheading edges are added to the merged route. In

fact, the process of adding the deadheading edges to the route is to use the shortest path when traversing from one required edge to another. The shortest paths can be obtained with the Floyd–Warshall algorithm. Repeated edges are encountered with the addition of the deadheading edges to the route. In this case, if it is possible, the deadheading edges on the route are converted into the required edges. The process of converting the deadheading edges into required edges is performed using the postpone mechanism described by Hertz et al. [31]. Thus, if the same edge is encountered multiple times on the route, the edge is serviced when it is traversed for the last time by the drone. The neighbor solution is then obtained by dividing the obtained route into k routes based on the drone capacities. Please note that drones can traverse directly between any two nodes on the graph, and this is actually the shortest path between these two nodes. But, so as to apply edge postpone and route-repairing operators, it is assumed that the drones can only traverse between the nodes of the components that view each other in the mechanism of edge postpone and route-repairing operators. Thus, different shortest paths are obtained using the Floyd–Warshall algorithm. So, the probability of deadheading on the required edges increases, which enables the effective use of the edge postpone and route-repairing operators.

Edge Direction and Smart Edge Direction Change Operator: The direction of an edge on a randomly selected route is changed using the edge direction change operator. The smart edge direction change operator is employed separately for each drone route. If the consecutive edges in the route contain common nodes, the direction of the edges is changed so that the common nodes are next to each other. These two operators are used as a single operator when selecting the neighbor search operator in the structure of the algorithm. However, both operators are not applied to the current solution at the same time. Between two operators, one will be selected at random.

Route-Repairing Operator: The aim of this operator is to obtain a new route with a shorter distance than the current one by ignoring the edge demands and drone capacity on the route. Therefore, this problem is similar to the RPP. The shorten procedure suggested by Hertz et al. [31] is used for this reason. However, the shorten procedure will only be applied to a randomly chosen drone route and a randomly chosen initial point. The steps of the shorten procedure are listed below:

- Step 1: Randomly select a drone route.
- Step 2: Add the deadheading edges to the selected route.
- Step 3: Randomly select a required edge on the route and make the selected required edge the starting point without altering the direction of the tour.
- Step 4: Run the postpone mechanism. If the same edge is encountered multiple times on the route, the edge is serviced when it is traversed for the last time by drone.
- Step 5: Run the reverse mechanism. In the reverse mechanism, if (i, j) is a required edge, it is checked whether there is a path $(i, j, \dots, k, i)$ in the tour that contains a non-required edge (k, i) . If there is a path with these features, the reversed version of this path $(i, k, \dots, j, i)$ should be used in the tour.
- Step 6: Remove the deadheading edges on the relevant drone route.

4.3. Constraint Handling Strategy

Drone capacities are allowed to be exceeded in the generating neighbor solution process. However, if the drone capacities are exceeded, the objective function value of the current solution is penalized. Therefore, the dynamic penalty function is used as a constraint handling strategy. The dynamic penalty function is developed to direct the search towards the global optimal solution using the information of infeasible solutions. At the same time, the search process uses the information from the recent iterations of the algorithm. The proposed dynamic penalty function is presented below:

$$\begin{aligned}
 f(s) &= \text{Objective function value of the solution} \\
 f'(s) &= \text{Penalized objective function value of the solution} \\
 p &= \text{The longest distance value in distance matrix} \\
 e &= \text{Excess capacity value} \\
 k &= \text{Penalty coefficient} \\
 k &= \begin{cases} 1, & \text{If the drone capacity is exceeded during the last two iterations} \\ 0.5, & \text{If the drone capacity is exceeded only the last iteration} \\ 0, & \text{If the drone capacity is not exceeded only the last iteration} \end{cases} \\
 f'(s) &= f(s) + (k \times p \times e) \tag{1}
 \end{aligned}$$

The $f(s)$ presented in Equation (1) represents the total distance traveled by the drones in solution s . On the other hand, $f'(s)$ represents the penalized objective function, which is the penalized version of $f(s)$. Therefore, $f'(s)$ is calculated by adding the penalty value to the total distance traveled by the drones in solution s . The main idea of the penalty function presented in Equation (1) is to penalize the objective function by adaptively updating the penalty coefficient. Namely, if the drone capacity is not exceeded in the last iteration, there is no penalty, but if it is exceeded 1 or 2 times, there is a gradually increasing penalty. Hence, even if exceeding drone capacities is allowed during the search phase, feasible solutions are eventually obtained since the solutions that exceed vehicle capacity are penalized. Furthermore, especially at high temperatures, diversification is provided, and quality solution regions is reached by using the information of infeasible solutions.

4.4. Cooling Schedule and Stopping Criteria

In the structure of the algorithm, a geometric cooling schedule is used to reduce the temperature in a controlled manner. The geometric cooling schedule is presented below:

T_i : Temperature value at iteration i : Cooling rate

$$T_i = \alpha \times T_{i-1} \tag{2}$$

In the geometric cooling schedule presented in Equation (2), the cooling rate is determined as $\alpha = 0.99$. Thus, the current temperature of the algorithm is gradually reduced, allowing for detailed exploration of the search space. The final temperature value is used as the stopping criterion of the algorithm. The final temperature value is determined as $T_f = 0.1$. If the current temperature value of the algorithm is less than or equal to the final temperature value, the algorithm is stopped, and the best obtained solution is presented.

4.5. Proposed Simulated Annealing Algorithm

In the previous subtitles, the components of the simulated annealing algorithm designed for the DARP-DD were discussed in detail. The pseudocode of the proposed simulated annealing algorithm is given below, and the flowchart of this algorithm is presented in Figure 5.

The working mechanism of the suggested simulated annealing algorithm is presented in Figure 5 to obtain high-quality solutions for the DARP-DD and MS-DARP-DD. In addition to the components of the algorithm that were designed, a strategy was created for the number of drones to be employed to improve the algorithm's flexibility during the search phase. The algorithm allows for operating with more drones than required. It is not necessary to use all of the drones provided as input to the algorithm. Some drones are not used if a better solution is obtained by using fewer drones than the number of drones provided as input to the algorithm. Thus, the algorithm can also determine the number of drones to be used.

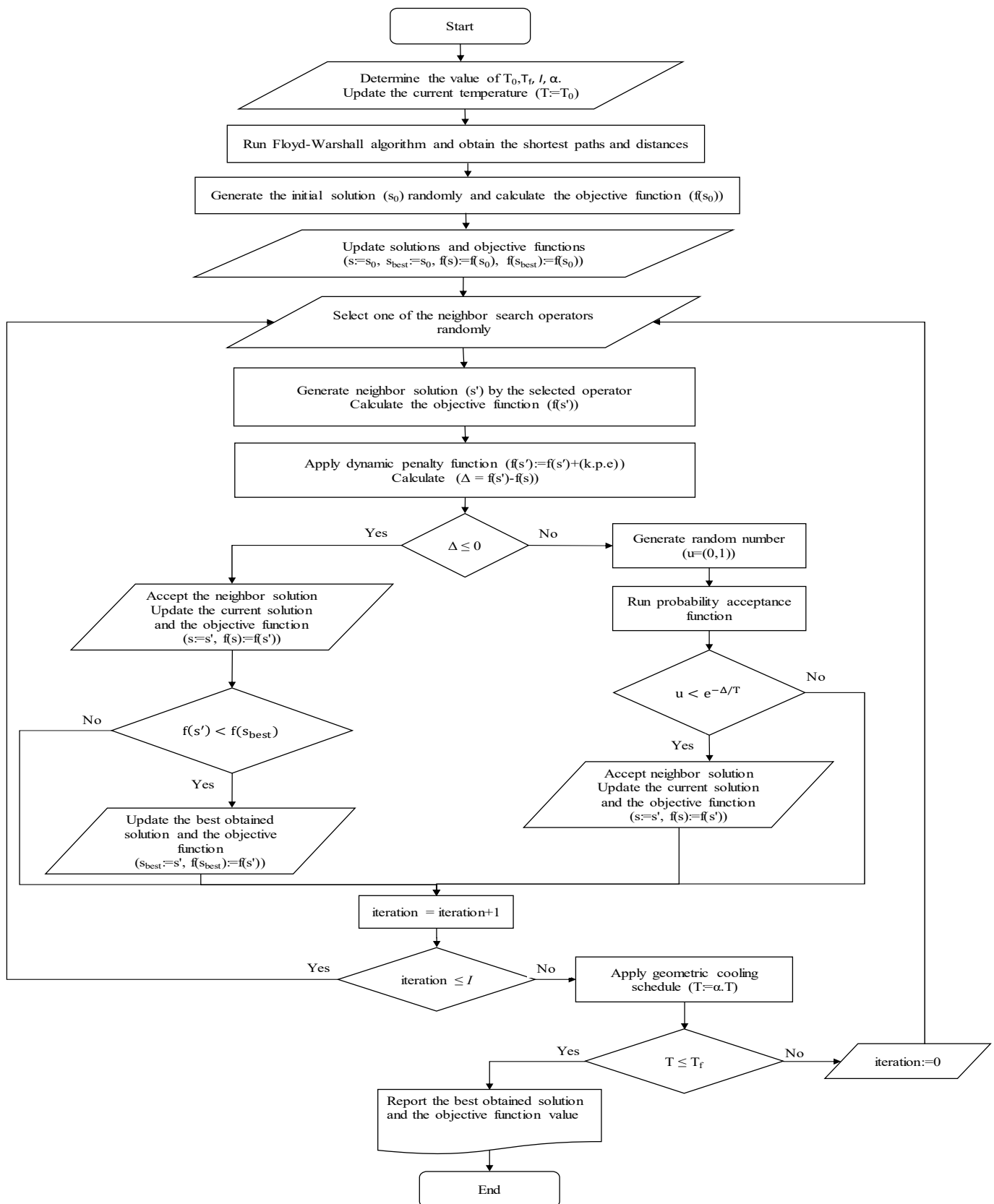


Figure 5. Flowchart of the proposed algorithm.

Definitions:

s_0 : initial solution	$f(s_0)$: objective function value of the initial solution
s : current solution	$f(s)$: objective function value of the current solution
s' : neighbor solution	$f(s')$: objective function value of the neighbor solution
s_{best} : the best obtained solution	$f(s_{best})$: objective function value of the best obtained solution
T_0 : initial temperature	p : the longest distance value in distance matrix
T : current temperature	k : penalty coefficient
T_f : final temperature	e : excess capacity value
α : cooling rate	I : number of iterations at each temperature

Pseudo-code of the Proposed Simulated Annealing Algorithm

1. Determine the value of T_0, T_f, I, α
2. Run Floyd–Warshall algorithm, obtain the shortest paths and distances between all pairs of nodes
3. Generate s_0 randomly and calculate $f(s_0)$
4. Update solutions, objective functions, and current temperature
($s := s_0, s_{best} := s_0, f(s) := f(s_0), f(s_{best}) := f(s_0), T := T_0$)
5. **Repeat**
6. **Repeat**
7. Select one of the neighbor search operators randomly
8. Generate neighbor solution (s') and calculate $f(s')$
9. Apply dynamic penalty function $f(s') := f(s') + (k \times p \times e)$
10. $\Delta = f(s') - f(s)$
11. **If** $\Delta \leq 0$;
12. Update the current solution $s := s', f(s) := f(s')$
13. **If** $f(s') < f(s_{best})$;
14. Update the best obtained solution $s_{best} := s', f(s_{best}) := f(s')$
15. **Else**;
16. Generate random number $u = (0, 1)$
17. **If** $u < e^{-\frac{\Delta}{T}}$;
18. Update the current solution $s := s', f(s) := f(s')$
19. **Until** (iteration $\geq I$)
20. Apply geometric cooling schedule $T := \alpha \times T$
21. **Until** ($T \leq T_f$)
22. Report the best obtained solution and objective function value $s_{best}, f(s_{best})$

5. Computational Results

The simulated annealing algorithm for the DARP was used to solve the CARP. The CARP was introduced to the literature by Golden and Wong [32], and it is known as one of the most challenging types of arc routing problems. The CARP is defined as obtaining vehicle routes that serve all the required edges without exceeding vehicle capacity to minimize total tour costs (distances). Firstly, the performance of the proposed algorithm on CARP test problems is analyzed. Then, the DARP-DD and MS-DARP-DD are solved by using the generated test problems. Finally, the performance of the designed components of the algorithm is presented.

5.1. Computational Results for CARP

The performance of the metaheuristic algorithm was tested using differently sized problems from the instance sets gdb by DeArmon [33], val by Benavent et al. [34], egl by Eglese [35], kshs by Kiuchi et al. [36], bmcv by Beullens et al. [37], eglL by Brandão and Eglese [38], and Hefei by Tang and Wang [39]. In particular, the largest and most complex problems from each instance set were used. The largest-sized problems in the gdb, val, egl, egl, kshs, bmcv, and eglL sets, and the Hefei-4 test problem from the Hefei set, in which a total of 1212 edges, 485 required edges, and 727 non-required edges were chosen. The

solution quality of the suggested algorithm is presented as the deviation value from the optimal or the best known solution in the literature.

$f(s)$: The objective function value obtained by the proposed algorithm
 $f(s^*)$: The objective function value of the optimal or the best known solution

$$\text{Deviation value (Gap)} = \left(\frac{f(s) - f(s^*)}{f(s^*)} \right) \quad (3)$$

The deviation value presented in Equation (3) is used as a performance indicator for the suggested method. On the other hand, the parameter values of the algorithm were determined as the initial temperature value (T_0), the final temperature value (T_f), and the cooling rate (α) values (T_0, T_f, α) = (100, 0.1, 0.99). The number of iterations (I) to be performed at each temperature was determined based on the problem size and is shown in Table 2.

Table 2. Number of iterations in simulated annealing algorithm.

Number of Required Edges	Number of Iterations
$E_R \leq 15$	50
$15 < E_R \leq 45$	200
$45 < E_R \leq 60$	500
$60 < E_R \leq 75$	750
$75 < E_R \leq 90$	1000
$90 < E_R \leq 200$	2000
$200 < E_R \leq 500$	5000

As presented in Table 2, the number of iterations is increased as the size of the problem increased in order to sufficiently explore the search space for each temperature value of the algorithm. The proposed algorithm for the CARP was coded using the Python 3.7 programming language and executed on an Intel Core i5 1.6 GHz computer with 8 GB RAM. The suggested algorithm was run 10 times for each test problem and the obtained results are presented in Table 3.

Table 3. Number of iterations in simulated annealing algorithm.

Test Problems	Required Edges (Total Edges)	Vehicles	Results of the Proposed Algorithm	Optimal/The Best Known Results	Gap	Computation Times (Seconds)
kshs-6*	15 (15)	3	10197	10197	-	4
gdb-5	26 (26)	6	377	377	-	30
gdb-21	33 (33)	6	156	156	-	34
gdb-22	44 (44)	8	200	200	-	40
gdb-23*	55 (55)	10	233	233	-	142
val-5c	65 (65)	5	474	474	-	291
val-5d	65 (65)	9	581	575	0.01	255
bmcv-d1	79 (98)	5	3235	3215	0.006	558
bmcv-f24*	86 (142)	4	3240	3210	0.009	794
val-10c	97 (97)	5	446	446	-	1104
val-10d*	97 (97)	10	530	525	0.009	987
egl-e4c	98 (98)	19	11631	11529	0.008	810
bmcv-c18	121 (133)	11	5640	5570	0.012	1102
bmcv-d18	121 (133)	6	4165	4165	-	1345
egl-s4a	190 (190)	19	12456	12216	0.019	2440
egl-s4b	190 (190)	27	16502	16201	0.018	2375
egl-s4c*	190 (190)	35	21207	20476	0.035	2345
egl-g2a	375 (375)	22	1128386	1088040	0.037	7404
egl-g2e*	375 (375)	42	1663216	1606079	0.035	7386
hefei-4	485 (1212)	28	767684	737730	0.040	9295

Table 3 summarizes the results of 20 differently sized test problems taken from the seven sets. The first column of Table 3 contains the names of the test problems. The names of the largest sized test problems in the set are denoted with an asterisk (kshs-6*, gdb-23*, bmcv-f24*, val-10d*, egl-s4c*, and eglL-g2e*). The number of required edges and the total number of edges in the relevant test problems are presented in the second column. The third column of the table shows the number of vehicles that serve the required edges. In the fourth column, the obtained objective function values for the test problems are presented. The next column contains the optimal or the best known objective function values. Deviation values (gap) are given in the fifth column. The last column shows the computation times reported in seconds.

The values of the optimal or best known results presented in the fifth column of Table 3 are based on findings from the literature. Martinez et al. [40] used a genetic algorithm for solving the CARP. In the structure of this algorithm, a random key was used for solution representation. The result of the kshs-6* problem presented in Table 3 was obtained in the study. Zhang et al. [41] proposed two divide-and-conquer algorithms called RCO-RDG-MAENS and RCO-SAHID to solve a large-sized CARP. In this study, the unified hybrid genetic search (UHGS) algorithm proposed by Vidal [42] was used to compare the algorithms, and the best known solution for the Hefei-4 problem, as presented in Table 3, was obtained by the UHGS algorithm. Chen et al. [43] proposed the hybrid metaheuristic approach (HMA) for effectively solving the CARP. In the structure of the algorithm, inversion, single insertion, double insertion, swap, two-opt, and merge-split operators were used as local refinement methods. The results of all other test problems presented in Table 3 (gdb-5, gdb-21, gdb-23*, val-5c, val-5d, bmcv-d1, bmcv-f24*, val-10c, val-10d*, egl-e4c, bmcv-c18, bmcv-d18, egl-s4a, egl-s4b, egl-s4c*, eglL-g2a, and eglL-g2e*) were obtained in the study. Since the number of vehicles to be used in the test problems is fixed, the same number of vehicles as presented in the third column of Table 3 was used in the aforementioned studies.

The test problems kshs-6 and gdb-23, which are the largest-sized problems in the kshs and gdb sets, were able to be solved optimally using the proposed algorithm. On the other hand, the optimal solutions could be obtained for the val-5c problem with 65 required edges and for the val-10c problem with 97 required edges. The optimal solution to the bmcv-d18 problem, which includes 121 required edges and a total of 133 edges, was achieved in 1345 s. This is the largest-sized problem for which the optimal solution could be obtained with the proposed algorithm. From these results, it is clear that the proposed algorithm is capable of reaching optimal solutions even for large-sized problems.

On the other hand, high-quality solutions were obtained for large-sized test problems in the egl set. For the egl-s4a and egl-s4b problems with 190 required edges, the solution with a deviation of about 1% from the best known result was reached in about 2300 s. The egl-s4c test problem, the largest-sized problem in the egl set, was solved with a 3.5% deviation from the best known result. For the eglL-g2a and egl-g2e problems in the EglL set with 375 required edges, high-quality solutions could be obtained in about 2 h with a deviation of about 3% from the best known result.

Comparing the proposed algorithm with the genetic algorithm proposed by Martinez et al. [40] for the kshs test problem, it is evident that both algorithms achieved the optimal solution. On the other hand, for the problems in the gdb, val, bmcv, egl, and eglL sets presented in Table 3, the proposed algorithm was compared with the HMA algorithm proposed by Chen et al. [43]. As a result, it was found that the proposed algorithm obtained the same results as the HMA for small-sized problems, approximately 0.9% away from the HMA for medium-sized problems, and approximately 3% away from the HMA for large-sized problems on average. Therefore, it is clear that the proposed algorithm performs very close to the algorithms that provide the optimal/the best known results for the CARP, demonstrating that it is a highly competitive algorithm.

In addition, high quality solution was obtained for the Hefei-4 test problem with a total of 1212 edges, 485 required edges, and 727 non-required edges in about 2.5 h with a

deviation of 4% from the best known result provided by Zhang et al. [41]. Therefore, when comparing the proposed solution approach with the UHGS algorithm, used by Zhang et al. [41] and proposed by Vidal [42], for the Hefei-4 problem, it is seen that the result obtained with the proposed approach is only 4% away from the best known result. This test problem is quite large-sized, and problem-specific heuristic algorithms are usually used to solve such challenging problems in the literature. For this reason, the suggested algorithm is successful and has the ability to reach high-quality solutions.

The performances of neighbor search operators were analyzed using the bmcv-d18 test instance, which is the large-sized test problem for which the optimal solution was obtained. As a result of the analysis, it is seen that the proposed algorithm improves the initial solution value 63 times in total and reaches the optimal solution. When the performances of the neighbor search operators were examined, k -route shorten improved the current solution 20 times, edge postpone 17 times, and edge direction and smart edge direction change 11 times, respectively. Therefore, it is seen that k -route shorten, edge postpone, edge direction, and smart edge direction change operators have high performance and make a great contribution to reaching high-quality solutions to the CARP.

5.2. Test Problems Generated for DARP

Seven differently sized test problems were generated for the DARP-DD. These test problems are used to test the performance of the proposed metaheuristic algorithm. The graphs are disconnected and consist of more than one component. The information about the test problems is presented in Table 4.

Table 4. Generated test problems.

Test Problems	Number of Components	Number of Nodes	Number of Required Edges	Number of Non-Required Edges
DARP-1	5	19	17	171
DARP-2	6	29	28	406
DARP-3	10	46	44	1035
DARP-4	11	64	64	2016
DARP-5	19	87	84	3741
DARP-6	17	98	100	4753
DARP-7	34	195	200	18,915

In Table 4, the first column presents the name of the test problem. Since the graphs to be used in the DARP-DD are disconnected, the number of components on the graph is given in the second column. The third column contains the number of nodes on the graph. The number of required and non-required edges on the graph is shown in the last two columns. The required edges on the graph can be curved or linear. The number of non-required edges in the graph is calculated by the number of nodes. The disconnected graph is converted to complete graph since the drone can directly traverse between all nodes on the graph. Since all node pairs are connected in a complete graph, this graph will have $(n \times (n - 1))/2$ non-required edges with n nodes. The largest-sized problem generated is DARP-7, which has 34 components, 200 required edges, and 18,915 non-required edges. There is detailed information about the test problems shown in Table 4 at <https://dataandresults.wixsite.com/darp>, accessed on 20 June 2023.

The algorithm can determine the number of drones that will service the required edges in the test problems. However, there is a need for the maximum number of drones to be used. The maximum number of drones to be used in the test problem can be calculated by the equation given below.

$i, j :$	Nodes
$d_{ij} :$	Energy demand value of required edges
$Q :$	Drone energy capacity
$n :$	The number of components
$d_{max} :$	The maximum energy demand value of deadheading edges
$k_{max} :$	Maximum number of drones

$$k_{max} = \left(\left(\sum_{(i,j) \in E_R} d_{ij} \right) + (n - 1) \times d_{max} \right) / Q \quad (4)$$

The maximum number of drones used in the solution of the problem is calculated by considering the total energy demand values of the required edges, the maximum energy demand value between pairs of nodes (while deadheading) on the graph, the smallest number of connections between components (number of components—1), and the drone energy capacity. The value of k_{max} is rounded up if the number of drones is not an integer.

5.3. Computational Results for DARP-DD

Seven different test problems were generated for the DARP-DD and the information about these test problems was detailed in the previous section. The performance of the proposed metaheuristic algorithm was tested in solving these problems.

The parameter values of the simulated annealing algorithm were determined as initial temperature value (T_0), final temperature value (T_f), and cooling coefficient (α) value (T_0, T_f, α) = (100, 0.1, 0.99). Since the initial solution of the algorithm is randomly generated, the initial temperature value is not varied according to the size of the problem. The number of iterations (I) to be operated at each temperature was determined based on the problem size and is shown in Table 5.

Table 5. The number of iterations in the SA algorithm.

Test Problems	Number of Required Edges	Number of Iterations
DARP-1	17	200
DARP-2	28	300
DARP-3	44	750
DARP-4	64	1250
DARP-5	84	2000
DARP-6	100	2500
DARP-7	200	5000

In order to sufficiently explore the search space for each temperature value of the algorithm, the number of iterations is increased as the size of the problem increases.

The URL heuristic algorithm was used to evaluate the quality of the results obtained with the proposed simulated annealing algorithm for the DARP-DD. The URL algorithm was proposed by Belenguer et al. [44] in the literature. In this algorithm, a giant tour is first created with path scanning with random link. Therefore, the five rules of the path scanning algorithm are not used in this algorithm, and it differs from the path scanning algorithm proposed by Golden et al. [45]. During the giant tour creation phase, path scanning with random link adds the feasible unserved edge that is nearest to the last node added. Then, the created giant tour is divided into vehicle routes using Ulusoy's splitting algorithm [46]. Moreover, the URL algorithm shows that 4 to 11 executions are enough to perform better than the path scanning algorithm [47].

The suggested simulated annealing algorithm and URL algorithm for the DARP-DD was run 10 times for each test problem and the best obtained results are presented in Table 6.

Table 6. The obtained results for the DARP-DD test problems.

Test Problems	Number of Components	Number of Required Edges	Number of Non-Required Edges	Proposed Simulated Annealing Algorithm			URL Algorithm	
				Number of Drones (Drone Capacity)	Results (Objective Function Values)	Computation Times (Seconds)	Number of Drones (Drone Capacity)	Results (Objective Function Values)
DARP-1	5	17	171	4 (500 J)	1204.46	30.4	5 (500 J)	1324.12
DARP-2	6	28	406	4 (1000 J)	2429.58	66.1	5 (1000 J)	2704.21
DARP-3	10	44	1035	6 (1000 J)	3287.91	190.7	6 (1000 J)	3661.5
DARP-4	11	64	2016	5 (1500 J)	3915.78	554.3	5 (1500 J)	4546.78
DARP-5	19	84	3741	7 (1500 J)	6238.37	810.6	8 (1500 J)	7296.33
DARP-6	17	100	4753	8 (1500 J)	6916.44	1319.8	10 (1500 J)	8033.11
DARP-7	34	200	18915	21 (1500 J)	20137.39	3651.3	24 (1500 J)	23845.77

Table 6 presents the results obtained for seven different DARP-DD test problems. The first four columns of Table 6 contain information about the test problem. The number of drones and the energy capacity of the drones in the test problems are given in the fifth column. Their energy capacities are presented in Joules and the number of drones used in each test problem is reported. In the sixth column, the obtained results for the DARP-DD are given, which are the total traveled distances by drones. The seventh column shows the computation times given in seconds. The last two columns in Table 6 present the results obtained with the URL algorithm.

Table 6 demonstrates that the proposed metaheuristic algorithm was capable of solving differently sized problems within reasonable computation times. For example, the DARP-2 problem with 6 components, 28 required edges, and 406 non-required edges was solved in about 66 s using four drones with an energy capacity of 1000 Joules. The total traveled distance by the drones was 2429.58 units for this problem. The DARP-7 problem includes 34 components, 200 required edges, and 18,915 non-required edges. For the DARP-7 problem, the solution was obtained in about 1 h by using 21 drones with an energy capacity of 1500 Joules. The total distance traveled by the drones was obtained as 20,137.39 units.

Examining the results presented in Table 6, it can be seen that the proposed simulated annealing algorithm is more successful and achieves better solutions than the URL algorithm. For example, for the DARP-1 problem, the total distance traveled with the proposed algorithm is 1204.46 units, while the total distance traveled with the URL algorithm is 1324.12 units. Therefore, the proposed algorithm provided a 9.9% better result for DARP-1 than the URL algorithm. For the large-sized DARP-7 problem, the proposed algorithm achieved an 18.4% better result than the URL algorithm. For the other test problems in Table 6, the proposed algorithm obtained an average of 14% higher quality results. Meanwhile, since the URL algorithm is a heuristic method, it is quite fast, and the results presented in Table 6 were obtained in less than three seconds for each test problem.

The DARP-1 and DARP-6 problems are taken into consideration to display the drone routes that correspond to the obtained solution shown in Table 6. The drone routes for the DARP-1 problem are shown in Figure 6 while the drone routes for the DARP-6 problem are presented in Appendix A.

As shown in Figure 6, 17 required edges were serviced by using four drones with an energy capacity of 500 Joules. The edges in each drone's route are shown in a different color. Dashed edges indicate deadheading on the relevant edge. Drone routes were obtained as follows:

First drone route (blue-colored route): d(depot – 10) – (10 – 8) – (8 – 7) – (7 – 9) – (9 – 10) – d(10 – depot).

Second drone route (red-colored route): d(depot – 16) – (16 – 15) – d(15 – 1) – (1 – 3) – d(3 – 6) – (6 – 5) – (5 – 4) – d(4 – depot).

Third drone route (green-colored route): d(depot – 2) – (2 – 1) – d(1 – 11) – (11 – 12) – d(12 – 6) – (6 – 4) – d(4 – depot).

Forth drone route (purple-colored route): $d(\text{depot} - 16) - (16 - 17) - (17 - 18) - (18 - 13) - (13 - 14) - (14 - 15) - d(15 - 2) - (2 - 3) - d(3 - \text{depot})$.

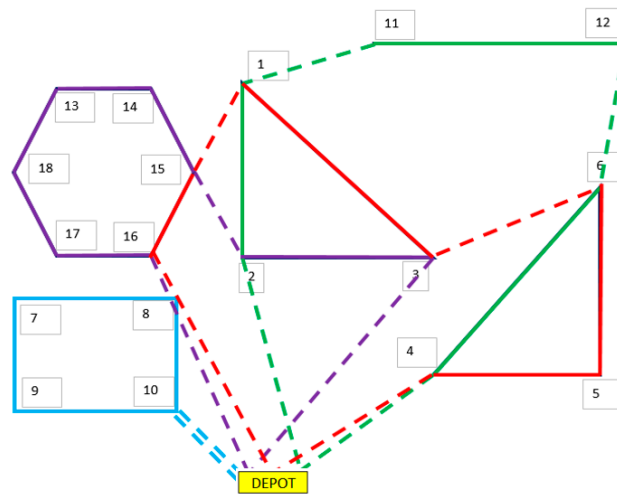


Figure 6. Drone routes obtained for DARP-DD-1.

Analyzing the drone routes reveals that some components on the graph are serviced by the same drone, while other components are serviced by different drones. In addition, because the energy capacity of the drones is consumed during deadheading, deadheading was made at the edges that give the shortest possible connection between the components. The capacity utilization rates of the drones are 75%, 99%, 96%, and 97%, respectively. The objective function value, the total distance traveled by the drones, was obtained as 1204.46 units.

5.4. Computational Results for MS-DARP-DD

Seven different test problems were generated for the DARP-DD, and the obtained results for these problems were presented in the previous section. The multiple service drone arc routing problem with deadheading demand (MS-DARP-DD), one of the special cases of the DARP-DD that involves servicing the required edges more than once, is discussed in this section. A multiple service condition was added to some required edges in the DARP-DD test problems. Thus, MS-DARP-DD test problems were generated. The proposed metaheuristic algorithm was applied to solve these problems. The parameter values of the algorithm were the same as for those determined in the DARP-DD.

The suggested algorithm was run 10 times for each test problem and the best obtained solutions are presented in Table 7.

Table 7. Obtained results for MS-DARP-DD test problems.

Test Problems	DARP-DD			MS-DARP-DD			
	Required Edges	Number of Drones (Drone Capacity)	Results (Objective Function Values)	Required Edges	Number of Drones (Drone Capacity)	Results (Objective Function Values)	Computation Times (Seconds)
DARP-1	17	4 (500 J)	1204.46	21	5 (500 J)	1417.3	32.5
DARP-2	28	4 (1000 J)	2429.58	32	5 (1000 J)	2707.22	70.2
DARP-3	44	6 (1000 J)	3287.91	51	7 (1000 J)	3719.41	210.3
DARP-4	64	5 (1500 J)	3915.78	69	5 (1500 J)	4067.02	580.4
DARP-5	84	7 (1500 J)	6238.37	91	8 (1500 J)	6649.84	834.9
DARP-6	100	8 (1500 J)	6916.44	107	9 (1500 J)	7123.38	1350.7
DARP-7	200	21 (1500 J)	20137.3	207	21 (1500 J)	20724.5	3750.9

Table 7 presents the obtained results for the DARP-DD, where each required edge must be serviced once, and the MS-DARP-DD, where some required edges must be serviced more than once. For example, in the DARP-2 problem for the DARP-DD, there are 28 required edges on the graph and each required edge must be serviced once. When this problem was solved with the suggested algorithm, the 28 required edges on the graph were serviced by four drones with an energy capacity of 1000 Joules and the total traveled distance was obtained as 2429.58 units. On the other hand, if the DARP-2 problem is considered for the MS-DARP-DD, there are 32 required edges on the graph with a multiple service condition providing service twice for edge (14 – 15), twice for edge (19 – 20), three times for edge (2 – 3), and once for the other required edges on the graph. The suggested algorithm was also used to solve this problem. The obtained results show that the 32 required edges on the graph were serviced by five drones with an energy capacity of 1000 Joules and the total traveled distance by the drones was obtained as 2707.22 units. The computation time of this problem is approximately 70 s. As shown in Table 7, since the MS-DARP-DD has a larger number of required edges than the DARP-DD, more drones can be used to service the required edges in most of the test problems. In addition, the total traveled distance increased in the MS-DARP-DD compared to the DARP-DD due to the increased number of required edges to be served in the graph.

When the MS-DARP-DD drone routes are analyzed, it is revealed that the required edges that need to be serviced more than once can be served by the same drones or by different drones. Consequently, multiple services do not have to be performed by the same drone.

The URL algorithm [44] was used to analyze the quality of the MS-DARP-DD results obtained by applying the proposed simulated annealing algorithm. Table 8 presents the results achieved using both methods.

Table 8. Comparison between the proposed simulated annealing and URL algorithms.

Test Problems	Number of Required Edges	Proposed Simulated Annealing Algorithm			URL Algorithm	
		Number of Drones (Drone Capacity)	Results (Objective Function Values)	Computation Times (Seconds)	Number of Drones (Drone Capacity)	Results (Objective Function Values)
DARP-1	17	5 (500 J)	1417.3	32.5	6 (500 J)	1551.81
DARP-2	28	5 (1000 J)	2707.22	70.2	6 (1000 J)	3048.65
DARP-3	44	7 (1000 J)	3719.41	210.3	8 (1000 J)	4264.57
DARP-4	64	5 (1500 J)	4067.02	580.4	6 (1500 J)	4702.6
DARP-5	84	8 (1500 J)	6649.84	834.9	9 (1500 J)	7864.6
DARP-6	100	9 (1500 J)	7123.38	1350.7	10 (1500 J)	8307.65
DARP-7	200	21 (1500 J)	20724.5	3750.9	25 (1500 J)	24718.4

Table 8 shows that the suggested simulated annealing algorithm outperforms the URL algorithm for all MS-DARP-DD instances. For example, in DARP-7, the largest-sized test problem, the objective function value obtained with the proposed algorithm is 20724.5, but the objective function value provided by the URL algorithm is 24718.4. Therefore, an approximately 20% better result was achieved for the DARP-7 instance with the proposed algorithm. Moreover, an average of 14% better results were obtained for the other test problems. Additionally, the number of drones used with the URL algorithm is higher for all test problems. In this table, no information is provided on the computing time of the URL algorithm, because it is a heuristic method, and the computing time is quite short. For each test problem, the computing time is less than three seconds.

The obtained drone routes for the DARP-1 problem when all required edges are serviced just once are shown in Figure 6. In the same problem, the drone routes obtained when (4 – 6) is serviced twice, edge (10 – 8) twice, edge (1 – 2) three times, and the other required edges once are presented in Figure 7. Furthermore, the drone routes for

the DARP-6 problem, where all required edges are serviced only once, are presented in Appendix A. In the same problem, the drone routes obtained when edge (60 – 61) is serviced twice, edge (70 – 71) five times, edge (95 – 96) three times, and the other required edges once are presented in Appendix B.

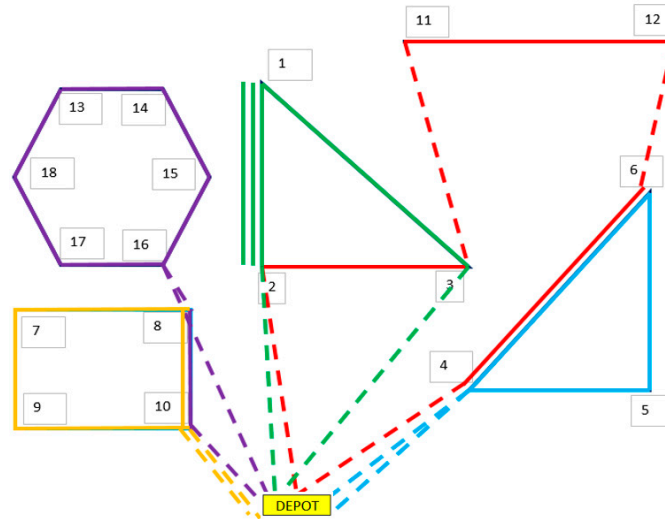


Figure 7. The obtained drone routes for MS-DARP-DD-1.

In Figure 7, 21 required edges were serviced by using five drones with an energy capacity of 500 Joules. The edges in each drone's route are shown in a different color. Dashed edges indicate deadheading on the relevant edge. Drone routes were obtained as follows:

First drone route (blue-colored route): $d(\text{depot} - 4) - (4 - 6) - (6 - 5) - (5 - 4) - d(4 - \text{depot})$.

Second drone route (red-colored route): $d(\text{depot} - 2) - (2 - 3) - d(3 - 11) - (11 - 12) - d(12 - 6) - (6 - 4) - d(4 - \text{depot})$.

Third drone route (green-colored route): $d(\text{depot} - 2) - (2 - 1) - (1 - 2) - (2 - 1) - (1 - 3) - d(3 - \text{depot})$.

Forth drone route (purple-colored route): $d(\text{depot} - 16) - (16 - 15) - (15 - 14) - (14 - 13) - (13 - 18) - (18 - 17) - (17 - 16) - d(16 - 8) - (8 - 10) - d(10 - \text{depot})$.

Fifth drone route (orange-colored route): $d(\text{depot} - 10) - (10 - 8) - (8 - 7) - (7 - 9) - (9 - 10) - d(10 - \text{depot})$.

When the obtained drone routes are evaluated, it is revealed that Drone-3 served the edge (1 – 2) three times in succession. The edge (4 – 6) was served by Drone-1 and Drone-2, whereas the edge (10 – 8) was served by Drone-4 and Drone-5. In other words, the multiple service demands of these edges were met by different drones. Examining the drone routes reveals that multiple service to the required edges need not be provided by the same drone. Multiple service can be carried out by the same drone if it is advantageous with regard to the traveled distance. On the other hand, the number of required edges on the graph increases in the case of multiple services and the number of drones increases to five. The capacity utilization rates of these drones are 79%, 97%, 97%, 97%, and 75%, respectively. The objective function value, the total traveled distance by the drones, was obtained as 1417.3 units.

5.5. Performance of the Simulated Annealing Algorithm Components

The performance of the components of the suggested algorithm was analyzed using the DARP-6 test problem. This problem has 17 components, 100 required edges, and 4753 non-required edges. The parameter values of the simulated annealing algorithm were determined as the initial temperature value (T_0), the final temperature value (T_f), the cooling rate (α), and the number of iteration (I) values $(T_0, T_f, \alpha, I) = (100, 0.1, 0.99, 2500)$.

When the DARP-6 problem was solved with the suggested algorithm, eight drones with an energy capacity of 1500 Joules were used in the obtained solution and the total distance traveled by the drones was 6916.44 units. The computation time of the algorithm for this problem is approximately 1319 s. The convergence graph of the suggested algorithm for the DARP-6 problem is presented in Figure 8.

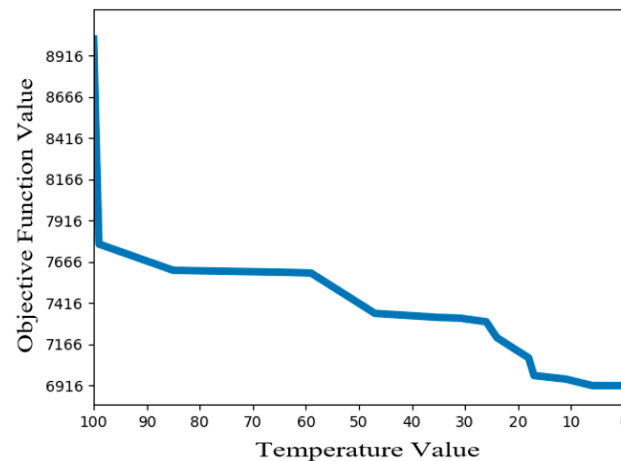


Figure 8. Convergence graph of the proposed algorithm for the DARP-6 problem.

Figure 8 shows the convergence graph of the suggested algorithm for DARP-6 based on the objective function values obtained between the initial and the final temperature values. Since the initial solution of the algorithm is randomly generated, the search process is initiated with solutions having very large objective function values. The objective function value of the first feasible solution obtained at the initial temperature values is 9017. However, the algorithm's well-designed components allowed for it to quickly converge to high-quality solution regions, and the objective function values were improved. For example, the obtained solution's objective function value was 7354 when the algorithm's temperature value was 47, and 7145 when the temperature value decreased to 21. At the end of the algorithm, a solution with a value of 6916.44 was reached owing to the intensification mechanism, which allowed for the algorithm to search more thoroughly for quality solution regions at lower temperature levels. Furthermore, the cooling rate in the geometric cooling schedule was set as close to 1 as possible. Thus, the algorithm was able to thoroughly search the solution space at each temperature value. This also enabled the algorithm to access the quality solution regions in the solution space.

A study was conducted on the impact of the dynamic penalty function, which is another component of the algorithm and used as a constraint-handling strategy, on the solution quality. When the DARP-6 problem was solved using the dynamic penalty function within the proposed algorithm's structure, the objective function value was obtained as 6916.44. On the other hand, the penalty coefficient value (k) was taken as 1 in the dynamic penalty function, thus obtaining a generic (fixed) penalty function. Then, the DARP-6 problem was solved using the generic penalty function within the proposed algorithm's structure. In this case, the objective function value was obtained as 7084.11. Therefore, it can be said that the dynamic penalty function improves solution quality by approximately 2.4% for DARP-6. Moreover, the same study was conducted for other test problems. On examining the results, it was found that the dynamic penalty function improves the solution quality by approximately 0.5% for DARP-1, DARP-2, and DARP-3; by approximately 1.2% for DARP-4 and DARP-5; and by approximately 2% for DARP-6 and DARP-7.

The initial temperature value and the number of iterations performed at each temperature were analyzed. The proposed algorithm was run for the solution of the DARP-6 problem with different initial temperatures and the number of iterations to be performed at each temperature. The obtained results of this experiment are presented in Table 9.

Table 9. Parameter analysis for the DARP-6 problem.

Initial Temperature	Number of Iteration	Minimum Gap	Average Gap	Computation Times (Seconds)
50	2000	0.012	0.025	850
	2500	0.008	0.016	1100
	3000	0.007	0.016	1423
	3500	0.007	0.015	1542
100	2000	0.005	0.015	1024
	2500	-	0.012	1319
	3000	-	0.01	1619
	3500	-	0.009	1825
150	2000	0.005	0.013	1044
	2500	-	0.011	1391
	3000	-	0.01	1698
	3500	-	0.009	1918
200	2000	0.004	0.011	1105
	2500	-	0.01	1458
	3000	-	0.009	1805
	3500	-	0.009	2008

DARP-6 was solved considering the algorithm's initial temperature values as 50, 100, 150, and 200 and the number of iterations at each temperature as 2000, 2500, 3000, and 3500, and the obtained results of these experiments are presented in Table 9. The algorithm was run 10 times for each initial temperature and the number of iterations. Table 9 presents the minimum deviation value, average deviation value, and computation times of each experiment. The deviation values were calculated using the objective function value of 6916.44 units as the basis. This objective function value, which is shown in the convergence graph, was obtained by setting the initial temperature value to 100 and the number of iterations to 2500.

Table 9 shows that the best obtained solution (the solution having objective function value of 6916.44 units) for the DARP-6 problem was reached when the initial temperature values were 100, 150, and 200 and the number of iterations at each temperature was 2500, 3000, and 3500. Consequently, the initial temperature value of 100 and the number of iterations at each temperature of 2500 provide the shortest computation time among the parameter values enabling the best solution to be obtained. That is why the initial temperature value was set to 100 and the number of iterations performed at each temperature was set to 2500.

Figure 9 shows the performance of the neighbor search operators used in the structure of the algorithm whose convergence graph is drawn in Figure 8. The performance of the neighbor search operators is evaluated with regard to the number of times they improve the current solution.

Regarding Figure 9, the edge postpone operator (F) demonstrates the highest performance with regard to the number of improvements to the current solution and contributes the greatest to obtaining high-quality solutions. The edge postpone operator (F) improved the current solution value 66 times. Another operator with high performance is the *k*-route shorten operator (E). The *k*-route shorten operator (E) contributes to improving the current solution 40 times. Apart from the *k*-route shorten (E) and edge postpone operators (F), the other operators that contribute the most to obtaining quality solutions are smart reverse (C), edge direction and smart edge direction change (G), and route-repairing operators (H).

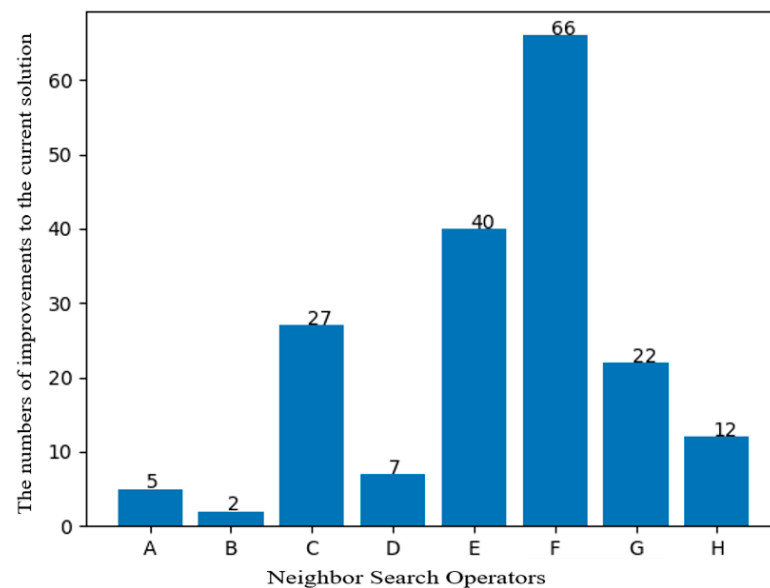


Figure 9. The number of times the neighbor search operators improve the current solution for the DARP-6 problem. The letters A, B, C, D, E, F, G, and H represent k -reverse, n -reverse, smart reverse, intra-route reverse, k -route shorten, edge postpone, edge direction and smart edge direction change, route-repairing operators, respectively.

The following comments can be made considering the temperature values at which the neighbor search operators improve the current solution. It is seen that the edge postpone operator (F), which has the highest performance in improving the solution, contributes to reaching quality solutions from the initial temperature to the final temperature of the algorithm. The k -route shorten operator (E), which is the second highest performing operator, has improved the current solution at several temperature values similar to the edge postpone operator (F). The final temperature value at which the k -route shorten operator (E) improves the current solution is approximately 10. The edge direction and smart edge direction change (G), smart reverse (C), and route-repairing operators (H) showed high performance, especially at the initial and final temperature values of the algorithm. The k -reverse (A) and n -reverse (B) operators, which are the two worst-performing operators, made a great contribution to the diversification mechanism of the algorithm since they caused major changes in the current solution. In addition, it might be concluded that the edge postpone (F) and k -route shorten (E) operators, which contribute to the improvement in the current solution at almost every temperature value of the algorithm, play a crucial role for the diversification and intensification mechanisms of the algorithm.

6. Conclusions and Future Works

This study focused on the routing of surveillance drones considering their energy capacities, which are consumed during both servicing and deadheading. The DARP-DD and its special case, the MS-DARP-DD, were discussed. The simulated annealing algorithm was used to solve these problems. The components of the simulated annealing algorithm, including solution representation, neighbor search operators, constraint handling strategy, and parameter values, were designed in accordance with the problems.

The solution representation was developed as a two-dimensional list based on the required edges. The k -route shorten, smart edge direction change, and smart reverse operators were introduced to the literature as neighbor search operators to be applied in other ARPs. In addition, it is the first time an edge postpone operator, which is employed to improve the tour of the RPP, was used as a neighbor search operator with merge and split procedure. A dynamic penalty function was developed and used in the structure of the algorithm as a constraint-handling strategy.

The proposed algorithm was initially applied to solve the CARP using 20 differently sized test problems. The optimal solution of the bmcv-d18 instance (121 required and 133 total edges) was obtained in 1345 s. For the Hefei-4 instance (485 required, 727 non-required, and 1212 total edges), the algorithm achieved a high-quality result within 2.5 h, deviating only 4% from the best known result. This demonstrates the algorithm's capability to solve large-sized problems efficiently.

Considering the ability of the proposed algorithm to achieve high-quality results for the CARP, the proposed algorithm was applied to solve the DARP-DD and MS-DARP-DD. Differently sized test problems were generated for the DARP-DD and MS-DARP-DD to test the performance of the proposed algorithm. The largest-sized test problem (DARP-7) had 34 components, 195 nodes, 200 required edges, and 18,915 non-required edges.

The computational results show that the proposed metaheuristic algorithm was capable of solving differently sized problems within reasonable computation times. Considering the DARP-7 problem for the DARP-DD, the solution was obtained in about 1 h by using 21 drones with an energy capacity of 1500 Joules. The total traveled distance by the drones was obtained as 20137.3 units. Furthermore, the obtained drone routes reveal that the capacity utilization rates of the drones are quite high, and deadheading was made at the edges that give the shortest possible connection between the components. These results demonstrate that the proposed algorithm can solve these particular instances within reasonable computing times. However, it is important to note that the scope of these results is confined to the defined problem sizes and scenarios. The algorithm's performance may vary with different problem characteristics.

Since the MS-DARP-DD has more edges than the DARP-DD, it needs more drones to service the required edges in most test problems. Furthermore, the total distance traveled by the drones and computation times increased in the MS-DARP-DD. When the drone routes for the MS-DARP-DD are analyzed, it is revealed that the required edges that need to be serviced more than once can be served by the same drones or by different drones. Therefore, this problem does not require the same drone to perform multiple services.

The performance of the neighbor search operators was evaluated with regard to the number of times that they improved the current solution. It was seen that the k -route shorten, edge postpone, edge direction and smart edge direction change, and smart reverse operators have high performance in the process of obtaining high-quality solutions to the algorithm.

This study presents a general framework for drone energy consumption. Additionally, the energy per unit distance can be adjusted based on drone type, usage purpose, or environmental conditions. On the other hand, the amount of energy consumed by drones when servicing the edges is not necessarily proportional to the distance traveled. As a result, this study outlines a flexible structure, and the parameter values can be easily changed. Researchers can apply the proposed algorithm by using specific energy consumption rates of the drones.

As a result of this study, two novel drone arc routing problems were discussed and an effective solution approach was developed to solve these problems. The developed solution approach and its components were designed to be extremely flexible. For this reason, the proposed algorithm can be used by researchers both for different conditions of the same problems and for the other arc routing problems. In future work, it may be useful to study different variants of the DARP-DD such as the multi-depot DARP-DD.

Author Contributions: The authors confirm their contributions to this paper as follows: study conception and design: I.A. and A.S.; computational experiments: I.A.; analysis and interpretation of results: I.A. and A.S.; draft manuscript preparation: I.A. and A.S. All authors have read and agreed to the published version of the manuscript.

Funding: The authors did not receive support from any organization for the submitted work.

Data Availability Statement: The data sets and test problems used in this research can be accessed on <https://dataandresults.wixsite.com/darp>, accessed on 20 June 2023.

Acknowledgments: This paper was produced from a doctoral thesis written by Islam Altin under the supervision of Aydin Sipahioglu.

Conflicts of Interest: The authors declare no conflict of interest.

Appendix A

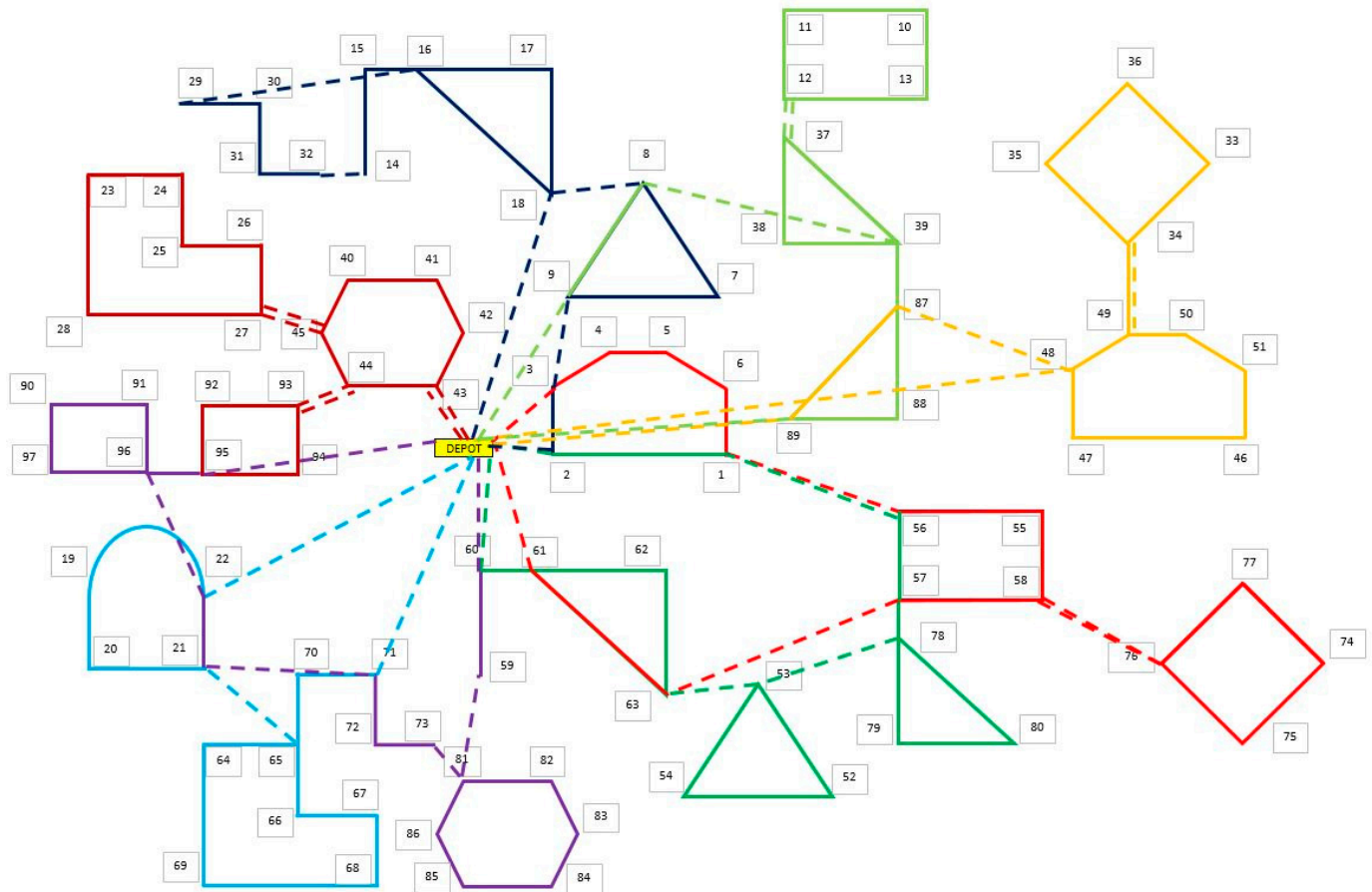


Figure A1. Drone routes for DARP-DD-6.

First drone route (red-colored route): $d(\text{depot} - 3) - (3 - 4) - (4 - 5) - (5 - 6) - (6 - 1) - d(1 - 56) - (56 - 55) - (55 - 58) - d(58 - 76) - (76 - 77) - (77 - 74) - (74 - 75) - (75 - 76) - d(76 - 58) - (58 - 57) - d(57 - 63) - (63 - 61) - d(61 - \text{depot})$.

Second drone route (green-colored route): $d(\text{depot} - 2) - (2 - 1) - d(1 - 56) - (56 - 57) - (57 - 78) - (78 - 79) - (79 - 80) - (80 - 78) - d(78 - 53) - (53 - 52) - (52 - 54) - (54 - 53) - d(53 - 63) - (63 - 62) - (62 - 61) - (61 - 60) - d(60 - \text{depot})$.

Third drone route (orange-colored route): $d(\text{depot} - 89) - (89 - 87) - d(87 - 48) - (48 - 47) - (47 - 46) - (46 - 51) - (51 - 50) - (50 - 49) - d(49 - 34) - (34 - 35) - (35 - 36) - (36 - 33) - (33 - 34) - (34 - 49) - (49 - 48) - d(48 - \text{depot})$.

Forth drone route (blue-colored route): $d(\text{depot} - 71) - (71 - 70) - (70 - 65) - (65 - 66) - (66 - 67) - (67 - 68) - (68 - 69) - (69 - 64) - (64 - 65) - d(65 - 21) - (21 - 20) - (20 - 19) - (19 - 22) - d(22 - \text{depot})$.

Fifth drone route (purple-colored route): $d(\text{depot} - 60) - (60 - 59) - d(59 - 81) - (81 - 86) - (86 - 85) - (85 - 84) - (84 - 83) - (83 - 82) - (82 - 81) - d(81 - 73) - (73 - 72) - (72 - 71) - d(71 - 21) - (21 - 22) - d(22 - 96) - (96 - 97) - (97 - 90) - (90 - 91) - (91 - 96) - (96 - 95) - d(95 - \text{depot})$.

Sixth drone route (dark red-colored route): $d(\text{depot} - 43) - (43 - 44) - d(44 - 93) - (93 - 94) - (94 - 95) - (95 - 92) - (92 - 93) - d(93 - 44) - (44 - 45) - d(45 - 27) - (27 - 26) - d(26 - \text{depot})$.

– (26 – 25) – (25 – 24) – (24 – 23) – (23 – 28) – (28 – 27) – d(27 – 45) – (45 – 40) – (40 – 41) – (41 – 42) – (42 – 43) – d(43 – depot).

Seventh drone route (light green-colored route): d(depot – 9) – (9 – 8) – d(8 – 39) – (39 – 37) – d(37 – 12) – (12 – 13) – (13 – 10) – (10 – 11) – (11 – 12) – d(12 – 37) – (37 – 38) – (38 – 39) – (39 – 87) – (87 – 88) – (88 – 89) – d(89 – depot).

Eighth drone route (dark blue-colored route): d(depot – 2) – (2 – 3) – d(3 – 9) – (9 – 7) – (7 – 8) – d(8 – 18) – (18 – 17) – (17 – 16) – d(16 – 29) – (29 – 30) – (30 – 31) – (31 – 32) – d(32 – 14) – (14 – 15) – (15 – 16) – (16 – 18) – d(18 – depot).

Appendix B

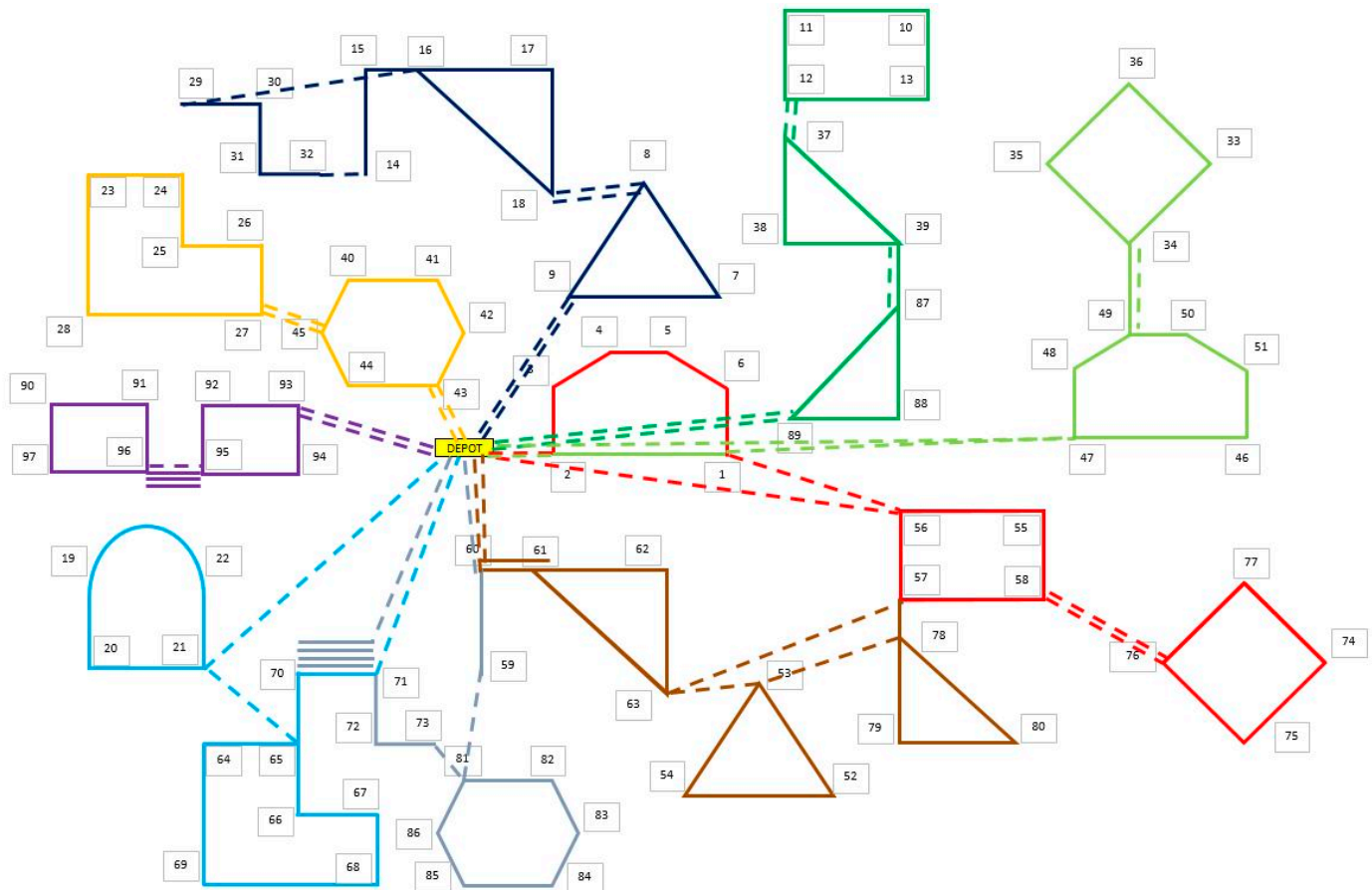


Figure A2. Drone routes for MS-DARP-DD-6.

First drone route (red-colored route): d(depot – 2) – (2 – 3) – (3 – 4) – (4 – 5) – (5 – 6) – (6 – 1) – d(1 – 56) – (56 – 55) – (55 – 58) – d(58 – 76) – (76 – 77) – (77 – 74) – (74 – 75) – (75 – 76) – d(76 – 58) – (58 – 57) – (57 – 56) – d(56 – depot).

Second drone route (green-colored route): d(depot – 89) – (89 – 87) – d(87 – 39) – (39 – 38) – (38 – 37) – d(37 – 12) – (12 – 13) – (13 – 10) – (10 – 11) – (11 – 12) – d(12 – 37) – (37 – 39) – (39 – 87) – (87 – 88) – (88 – 89) – d(89 – depot).

Third drone route (orange-colored route): d(depot – 43) – (43 – 42) – (42 – 41) – (41 – 40) – (40 – 45) – d(45 – 27) – (27 – 26) – (26 – 25) – (25 – 24) – (24 – 23) – (23 – 28) – (28 – 27) – d(27 – 45) – (45 – 44) – (44 – 43) – d(43 – depot).

Forth drone route (blue-colored route): d(depot – 71) – (71 – 70) – (70 – 65) – (65 – 66) – (66 – 67) – (67 – 68) – (68 – 69) – (69 – 64) – (64 – 65) – d(65 – 21) – (21 – 22) – (22 – 19) – (19 – 20) – (20 – 21) – d(21 – depot).

Fifth drone route (purple-colored route): $d(\text{depot} - 93) - (93 - 94) - (94 - 95) - (95 - 96) - (96 - 91) - (91 - 90) - (90 - 97) - (97 - 96) - (96 - 95) - (95 - 96) - d(96 - 95) - (95 - 92) - (92 - 93) - d(93 - \text{depot})$.

Sixth drone route (dark blue-colored route): $d(\text{depot} - 9) - (9 - 7) - (7 - 8) - d(8 - 18) - (18 - 16) - d(16 - 29) - (29 - 30) - (30 - 31) - (31 - 32) - d(32 - 14) - (14 - 15) - (15 - 16) - (16 - 17) - (17 - 18) - d(18 - 8) - (8 - 9) - d(9 - \text{depot})$.

Seventh drone route (light green-colored route): $d(\text{depot} - 47) - (47 - 46) - (46 - 51) - (51 - 50) - (50 - 49) - d(49 - 34) - (34 - 35) - (35 - 36) - (36 - 33) - (33 - 34) - (34 - 49) - (49 - 48) - (48 - 47) - d(47 - 1) - (1 - 2) - d(2 - \text{depot})$.

Eighth drone route (brown-colored route): $d(\text{depot} - 60) - (60 - 61) - (61 - 62) - (62 - 63) - d(63 - 53) - (53 - 54) - (54 - 52) - (52 - 53) - d(53 - 78) - (78 - 79) - (79 - 80) - (80 - 78) - (78 - 57) - d(57 - 63) - (63 - 61) - (61 - 60) - d(60 - \text{depot})$.

Ninth drone route (gray-colored route): $d(\text{depot} - 60) - (60 - 59) - d(59 - 81) - (81 - 82) - (82 - 83) - (83 - 84) - (84 - 85) - (85 - 86) - (86 - 81) - d(81 - 73) - (73 - 72) - (72 - 71) - (71 - 70) - (70 - 71) - (71 - 70) - (70 - 71) - d(71 - \text{depot})$.

References

- Corberán, Á.; Eglese, R.; Hasle, G.; Plana, I.; Sanchis, J.M. Arc Routing Problems: A Review of the Past, Present, and Future. *Networks* **2021**, *77*, 88–115. [\[CrossRef\]](#)
- Poikonen, S.; Campbell, J.F. Future Directions in Drone Routing Research. *Networks* **2021**, *77*, 116–126. [\[CrossRef\]](#)
- Macrina, G.; Pugliese, L.D.P.; Guerriero, F.; Laporte, G. Drone-Aided Routing: A Literature Review. *Transp. Res. Part C Emerg. Technol.* **2020**, *120*, 102762. [\[CrossRef\]](#)
- Otto, A.; Agatz, N.; Campbell, J.; Golden, B.; Pesch, E. Optimization Approaches for Civil Applications of Unmanned Aerial Vehicles (UAVs) or Aerial Drones: A Survey. *Networks* **2018**, *72*, 411–458. [\[CrossRef\]](#)
- Gohari, A.; Ahmad, A.B.; Rahim, R.B.A.; Supa'at, A.S.M.; Abd Razak, S.; Gismalla, M.S.M. Involvement of Surveillance Drones in Smart Cities: A Systematic Review. *IEEE Access* **2022**, *10*, 56611–56628. [\[CrossRef\]](#)
- Kinaneva, D.; Hristov, G.; Raychev, J.; Zahariev, P. Early Forest Fire Detection Using Drones and Artificial Intelligence. In Proceedings of the 2019 42nd International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), Opatija, Croatia, 20–24 May 2019; pp. 1060–1065.
- Paucar, C.; Morales, L.; Pinto, K.; Sánchez, M.; Rodríguez, R.; Gutierrez, M.; Palacios, L. Use of Drones for Surveillance and Reconnaissance of Military Areas. In *Developments and Advances in Defense and Security*; Rocha, Á., Guarda, T., Eds.; Smart Innovation, Systems and Technologies; Springer International Publishing: Cham, Switzerland, 2018; Volume 94, pp. 119–132, ISBN 978-3-319-78604-9.
- Loukinas, P. Drones for Border Surveillance: Multipurpose Use, Uncertainty and Challenges at EU Borders. *Geopolitics* **2022**, *27*, 89–112. [\[CrossRef\]](#)
- Lu, F.; Jiang, R.; Bi, H.; Gao, Z. Order Distribution and Routing Optimization for Takeout Delivery under Drone–Rider Joint Delivery Mode. *J. Theor. Appl. Electron. Commer. Res.* **2024**, *19*, 774–796. [\[CrossRef\]](#)
- Eskandaripour, H.; Boldsaikhan, E. Last-Mile Drone Delivery: Past, Present, and Future. *Drones* **2023**, *7*, 77. [\[CrossRef\]](#)
- Coutinho, W.P.; Battarra, M.; Fliege, J. The Unmanned Aerial Vehicle Routing and Trajectory Optimisation Problem, a Taxonomic Review. *Comput. Ind. Eng.* **2018**, *120*, 116–128. [\[CrossRef\]](#)
- Toth, P.; Vigo, D. (Eds.) *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed.; Society for Industrial and Applied Mathematics: Philadelphia, PA, USA, 2014; pp. i–xviii, ISBN 978-1-61197-358-7.
- Dror, M. *Arc Routing: Theory, Solutions and Applications*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2012.
- Campbell, J.F.; Corberán, Á.; Plana, I.; Sanchis, J.M. Drone Arc Routing Problems. *Networks* **2018**, *72*, 543–559. [\[CrossRef\]](#)
- Xu, B.; Zhao, K.; Luo, Q.; Wu, G.; Pedrycz, W. A GV-Drone Arc Routing Approach for Urban Traffic Patrol by Coordinating a Ground Vehicle and Multiple Drones. *Swarm Evol. Comput.* **2023**, *77*, 101246. [\[CrossRef\]](#)
- Kirlik, G.; Sipahioglu, A. Capacitated Arc Routing Problem with Deadheading Demands. *Comput. Oper. Res.* **2012**, *39*, 2380–2394. [\[CrossRef\]](#)
- Xie, A.; Miyagawa, K.; Wu, W.; Yagiura, M. Shizuoka University, Japan A Heuristic Algorithm for the Drone Rural Postman Problem. *J. Ind. Manag. Optim.* **2023**, *20*, 1951–1966. [\[CrossRef\]](#)
- Campbell, J.F.; Corberán, Á.; Plana, I.; Sanchis, J.M.; Segura, P. Solving the Length Constrained K-Drones Rural Postman Problem. *Eur. J. Oper. Res.* **2021**, *292*, 60–72. [\[CrossRef\]](#)
- Katila, C.J.; Di Gianni, A.; Buratti, C.; Verdone, R. Routing Protocols for Video Surveillance Drones in IEEE 802.11 s Wireless Mesh Networks. In Proceedings of the 2017 European Conference on Networks and Communications (EuCNC), Oulu, Finland, 12–15 June 2017; pp. 1–5.
- Cabuk, U.C.; Tosun, M.; Dagdeviren, O.; Ozturk, Y. Modeling Energy Consumption of Small Drones for Swarm Missions. *IEEE Trans. Intell. Transport. Syst.* **2024**, *1*–14. [\[CrossRef\]](#)
- Luo, Y.; Chen, Y. Energy-Aware Dynamic 3D Placement of Multi-Drone Sensing Fleet. *Sensors* **2021**, *21*, 2622. [\[CrossRef\]](#)

22. Zhang, J.; Campbell, J.F.; Sweeney II, D.C.; Hupman, A.C. Energy Consumption Models for Delivery Drones: A Comparison and Assessment. *Transp. Res. Part D Transp. Environ.* **2021**, *90*, 102668. [\[CrossRef\]](#)
23. Kirkpatrick, S.; Gelatt, C.D.; Vecchi, M.P. Optimization by Simulated Annealing. *Science* **1983**, *220*, 671–680. [\[CrossRef\]](#)
24. Černý, V. Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm. *J. Optim. Theory Appl.* **1985**, *45*, 41–51. [\[CrossRef\]](#)
25. Garfinkel, R.S.; Webb, I.R. On Crossings, the Crossing Postman Problem, and the Rural Postman Problem. *Networks* **1999**, *34*, 173–180. [\[CrossRef\]](#)
26. Dewil, R.; Vansteenwegen, P.; Cattrysse, D. A Review of Cutting Path Algorithms for Laser Cutters. *Int. J. Adv. Manuf. Technol.* **2016**, *87*, 1865–1884. [\[CrossRef\]](#)
27. Corberán, A.; Plana, I.; Sanchis, J.M. A Branch & Cut Algorithm for the Windy General Routing Problem and Special Cases. *Networks* **2007**, *49*, 245–257. [\[CrossRef\]](#)
28. Campbell, J.; Corberán, Á.; Plana, I.; Sanchis, J.M.; Segura, P. Polyhedral Analysis and a New Algorithm for the Length Constrained K-Drones Rural Postman Problem. *Comput. Optim. Appl.* **2022**, *83*, 67–109. [\[CrossRef\]](#)
29. Abubakar, A.I.; Ahmad, I.; Omeke, K.G.; Ozturk, M.; Ozturk, C.; Abdel-Salam, A.M.; Mollel, M.S.; Abbasi, Q.H.; Hussain, S.; Imran, M.A. A Survey on Energy Optimization Techniques in UAV-Based Cellular Networks: From Conventional to Machine Learning Approaches. *Drones* **2023**, *7*, 214. [\[CrossRef\]](#)
30. Tang, K.; Mei, Y.; Yao, X. Memetic Algorithm with Extended Neighborhood Search for Capacitated Arc Routing Problems. *IEEE Trans. Evol. Comput.* **2009**, *13*, 1151–1166. [\[CrossRef\]](#)
31. Hertz, A.; Laporte, G.; Hugo, P.N. Improvement Procedures for the Undirected Rural Postman Problem. *INFORMS J. Comput.* **1999**, *11*, 53–62. [\[CrossRef\]](#)
32. Golden, B.L.; Wong, R.T. Capacitated Arc Routing Problems. *Networks* **1981**, *11*, 305–315. [\[CrossRef\]](#)
33. DeArmon, J.S. A Comparison of Heuristics for the Capacitated Chinese Postman Problem. Ph.D. Thesis, University of Maryland, College Park, MD, USA, 1981.
34. Benavent, E.; Campos, V.; Corberán, A.; Mota, E. The Capacitated Chinese Postman Problem: Lower Bounds. *Networks* **1992**, *22*, 669–690. [\[CrossRef\]](#)
35. Eglese, R.W. Routeing Winter Gritting Vehicles. *Discret. Appl. Math.* **1994**, *48*, 231–244. [\[CrossRef\]](#)
36. Kiuchi, M.; Shinano, Y.; Hirabayashi, R.; Saruwatari, Y. An Exact Algorithm for the Capacitated Arc Routing Problem Using Parallel Branch and Bound Method. In *Spring National Conference of the Operational Research Society of Japan*; INFORMS: Catonsville, MD, USA, 1995; pp. 28–29.
37. Beullens, P.; Muyldermans, L.; Cattrysse, D.; Van Oudheusden, D. A Guided Local Search Heuristic for the Capacitated Arc Routing Problem. *Eur. J. Oper. Res.* **2003**, *147*, 629–643. [\[CrossRef\]](#)
38. Brandão, J.; Eglese, R. A Deterministic Tabu Search Algorithm for the Capacitated Arc Routing Problem. *Comput. Oper. Res.* **2008**, *35*, 1112–1126. [\[CrossRef\]](#)
39. Tang, K.; Wang, J.; Li, X.; Yao, X. A Scalable Approach to Capacitated Arc Routing Problems Based on Hierarchical Decomposition. *IEEE Trans. Cybern.* **2016**, *47*, 3928–3940. [\[CrossRef\]](#) [\[PubMed\]](#)
40. Martinez, C.; Loiseau, I.; Resende, M.G.; Rodriguez, S. BRKGA Algorithm for the Capacitated Arc Routing Problem. *Electron. Notes Theor. Comput. Sci.* **2011**, *281*, 69–83. [\[CrossRef\]](#)
41. Zhang, Y.; Mei, Y.; Zhang, B.; Jiang, K. Divide-and-Conquer Large Scale Capacitated Arc Routing Problems with Route Cutting off Decomposition. *Inf. Sci.* **2021**, *553*, 208–224. [\[CrossRef\]](#)
42. Vidal, T. Node, Edge, Arc Routing and Turn Penalties: Multiple Problems—One Neighborhood Extension. *Oper. Res.* **2017**, *65*, 992–1010. [\[CrossRef\]](#)
43. Chen, Y.; Hao, J.-K.; Glover, F. A Hybrid Metaheuristic Approach for the Capacitated Arc Routing Problem. *Eur. J. Oper. Res.* **2016**, *253*, 25–39. [\[CrossRef\]](#)
44. Belenguer, J.-M.; Benavent, E.; Lacomme, P.; Prins, C. Lower and Upper Bounds for the Mixed Capacitated Arc Routing Problem. *Comput. Oper. Res.* **2006**, *33*, 3363–3383. [\[CrossRef\]](#)
45. Golden, B.L.; DeArmon, J.S.; Baker, E.K. Computational Experiments with Algorithms for a Class of Routing Problems. *Comput. Oper. Res.* **1983**, *10*, 47–59. [\[CrossRef\]](#)
46. Ulusoy, G. The Fleet Size and Mix Problem for Capacitated Arc Routing. *Eur. J. Oper. Res.* **1985**, *22*, 329–337. [\[CrossRef\]](#)
47. Corberán, Á.; Laporte, G. (Eds.) *Arc Routing: Problems, Methods, and Applications*; Society for Industrial and Applied Mathematics: Philadelphia, PA, USA, 2015; pp. i–xxii, ISBN 978-1-61197-366-2.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.